

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении лабораторных работ №1-№5
по дисциплине «Теория графов»
на тему:
«Реализация различных алгоритмов на графах»
Вариант 2

Выполнил студент
группы 5130201/40002

Алексеев Л. К.

Проверил
преподаватель

Востров А. В.

«_____» _____ 2026г.

Содержание

Введение	3
1 Математическое описание	5
1.1 Связный ациклический граф	5
1.2 Распределение Чампернауна	5
1.3 Метод Шимбелла	6
1.4 Поиск количества маршрутов от одной вершины до другой	6
1.5 Точки сочленения графа	6
1.6 Алгоритм Дейкстры	7
1.7 Алгоритм Форда-Фалкерсона	8
1.8 Алгоритм поиска заданного потока минимальной стоимости	9
1.9 Матричная теорема Кирхгофа	11
1.10 Алгоритм Борувки	11
1.11 Код Прюфера	12
1.12 Алгоритм нахождения минимальной раскраски графа	12
1.13 Эйлеров граф	12
1.14 Разрез графа	14
1.15 Фундаментальная система разрезов	14
2 Особенности реализации	15
2.1 Структуры данных	15
2.2 Связный ациклический граф	16
2.3 Распределение Чампернауна	17
2.4 Метод Шимбелла	18
2.5 Точки сочленения графа	18
2.6 Алгоритм Дейкстры	19
2.7 Алгоритм Форда-Фалкерсона (Эдмондс-Карп)	20
2.8 Алгоритм поиска заданного потока минимальной стоимости	21
2.9 Матричная теорема Кирхгофа	23
2.10 Алгоритм Борувки	24
2.11 Код Прюфера	25
2.12 Алгоритм нахождения минимальной раскраски графа	29
2.13 Эйлеров граф	30
2.14 Разрез графа	32
2.15 Фундаментальная система разрезов	33
2.16 Меню программы (main.cpp)	34
3 Результаты работы	50
Заключение	61
Список литературы	63

Введение

В отчёте представлено описание лабораторных работ №1–№5 по дисциплине «Теория графов». В ходе выполнения работы необходимо реализовать программу, которая строит связный ациклический ориентированный граф в соответствии с заданным распределением (распределение Чампернауна), а также реализовать требуемые методы и алгоритмы над графом.

Лабораторная работа №1

1. Сформировать случайным образом связный ациклический ориентированный граф в соответствии с распределением Чампернауна (параметры распределения задаются как константы) с вводимым пользователем количеством вершин. Генерация выполняется по степеням вершин. В зависимости от дальнейших заданий используется ориентированный или неориентированный граф, полученный из ориентированного.
2. Вычислить эксцентриситеты вершин, определить центр графа и диаметральные вершины.
3. Реализовать метод Шимбелла для весовой матрицы, сгенерированной в соответствии с заданным распределением. В ответе представить минимальный и/или максимальный путь в виде матрицы. Весовая матрица генерируется с возможностью выбора пользователем: только положительные веса, только отрицательные или смешанные.
4. Определить возможность построения маршрута от одной заданной точки до другой (вершины вводит пользователь) и указать количество таких маршрутов.
5. Реализовать меню для выбора действий.

Лабораторная работа №2

1. Для графов, сгенерированных в первой работе, реализовать алгоритм поиска точек сочленения.
2. Сгенерировать весовую матрицу (положительную или с отрицательными весами – на выбор пользователя) и найти кратчайший путь для двух выбранных пользователем вершин с помощью классического алгоритма Дейкстры. Результат содержит расстояние, сам путь в виде последовательности вершин и вектор расстояний.
3. Сравнить скорость работы реализованных алгоритмов по количеству итераций.

Лабораторная работа №3

1. На основе сгенерированного ориентированного графа случайным образом получить матрицы пропускных способностей и стоимости.
2. Для полученного графа найти максимальный поток по алгоритму Форда–Фалкерсона.
3. Вычислить поток минимальной стоимости. Величина потока задаётся как $\lceil \frac{2}{3} \cdot \max \rceil$, где $\max$ – значение максимального потока. Для этого использовать ранее реализованный алгоритм Дейкстры.

Лабораторная работа №4

1. Для неориентированных графов, сгенерированных в первой работе, найти число остовных деревьев с помощью матричной теоремы Кирхгофа.
2. Построить минимальный по весу остов для сгенерированного неориентированного взвешенного графа, используя алгоритм Борувки. Полученный остов закодировать с помощью кода Прюфера и декодировать его, обязательно сохраняя веса.
3. Реализовать на исходном графе и полученном остове (по выбору пользователя) алгоритм нахождения минимальной раскраски графа.

Лабораторная работа №5

1. Для неориентированных графов, сгенерированных в первой работе, проверить, является ли граф эйлеровым. Если нет – модифицировать граф (логировать изменения) и построить эйлеров цикл.
2. Используя кратчайший остов, построенный в лабораторной работе №4, для получить фундаментальную систему разрезов (вывести на экран). На основе фундаментальных разрезов с помощью операции симметрической разности получить остальные разрезы (выбранные разрезы вводит пользователь).

1 Математическое описание

1.1 Связный ациклический граф

Графом $G(V, E)$ называется совокупность двух множеств — непустого множества V (множества вершин) и множества E двухэлементных подмножеств множества V (E — множество рёбер):

$$G(V, E) = \langle V; E \rangle, \quad V \neq \emptyset, \quad E \subset 2^V \wedge \forall e \in E (|e| = 2).$$

Маршрутом в графе называется чередующаяся последовательность вершин и рёбер, начинающаяся и кончающаяся вершиной, в которой любые два соседних элемента инцидентны, причём однородные элементы (вершины, рёбра) через один смежны или совпадают. Если все рёбра различны, то маршрут называется цепью. Если все вершины (а значит, и рёбра) различны, то маршрут называется простой цепью. Замкнутая цепь называется циклом, замкнутая простая цепь называется простым циклом. Число циклов в графе G обозначается $z(G)$. Граф без циклов называется ациклическим.

Говорят, что две вершины в графе связаны, если существует соединяющая их (простая) цепь. Граф, в котором все вершины связаны, называется связным.

1.2 Распределение Чампернауна

Распределение Чампернауна — это непрерывное распределение вероятностей, применяемое для моделирования данных.

Для генерации случайных чисел, имеющих распределение Чампернауна, используется метод обратного преобразования. Если r — равномерно распределённое на интервале $(0, 1)$ случайное число, то величина

$$x = \mu + \frac{1}{\alpha} \ln \operatorname{tg} \left(\frac{\pi r}{2} \right)$$

подчиняется распределению Чампернауна с параметрами μ и α . В данной работе используется именно эта формула для генерации случайных чисел.

На рис. 1 представлена гистограмма на 1000 значений для распределения Чампернауна для констант $mu = 1$ и $alpha = 1.57$ используемых в программе.

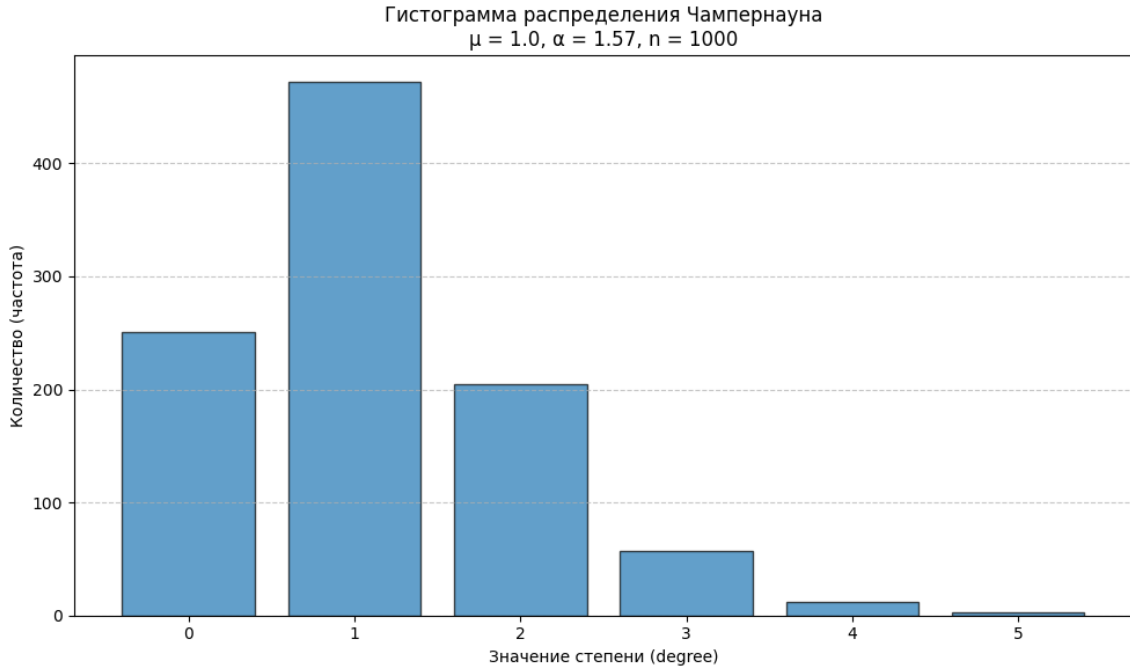


Рис. 1: Гистограмма распределения Чампернауна для констант $\mu = 1$, и $\alpha = 1.57$

1.3 Метод Шимбелла

Метод Шимбелла позволяет находить кратчайшие или максимальные пути между вершинами, состоящие из заданного количества рёбер. Операции в методе:

1. Умножение двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме:
 $a \cdot b = b \cdot a \Rightarrow a + b = b + a,$
 $a \cdot 0 = 0 \cdot a \Rightarrow a + 0 = 0.$
2. Операция сложения двух величин a и b заменяется выбором минимального (максимального) элемента, то есть:
 $a + b = b + a = \min(\max)\{a, b\}.$

С помощью операций Шимбелла длины кратчайших или максимальных путей между всеми вершинами определяются возведением в степень весовой матрицы, содержащей веса рёбер. Степень равна количеству рёбер в искомом пути. Сложность метода Шимбелла составляет $O(kn^3)$, где n — количество вершин, а k — длина пути (количество рёбер).

1.4 Поиск количества маршрутов от одной вершины до другой

Пусть дан простой граф $G = (V, A)$, матрица смежности которого есть $A = (a_{ij})_{n \times n}$, где $a_{ij} = 1 \Leftrightarrow (i, j) \in A$. Матрица $R(G) = E \vee A \vee A^2 \vee \dots \vee A^{n-1}$ есть матрица достижимости, где E — единичная матрица. Количество маршрутов длины k между вершинами определяется элементами A^k . Сложность поиска количества маршрутов длины k между вершинами через возведение матрицы смежности в степень составляет $O(kn^3)$, где n — число вершин.

1.5 Точки сочленения графа

Вершина графа называется точкой сочленения, или разделяющей вершиной, если её удаление увеличивает число компонент связности. Пусть $G(V, E)$ — связный граф и $v \in V$. Тогда следующие утверждения эквивалентны:

1. v — точка сочленения.
2. $\exists u, w \in V (u \neq w \ \& \ \forall \langle u, w \rangle_G (v \in \langle u, w \rangle))$.
3. $\exists U, W (U \cap W = \emptyset \ \& \ U \cup W = V - v \ \& \ \forall u \in U, w \in W (\forall \langle u, w \rangle_G (v \in \langle u, w \rangle_G)))$.

Для их поиска используется модифицированный обход в глубину (DFS), в ходе которого для каждой вершины вычисляются глубина входа в дерево обхода и наименьшая глубина, достижимая из её поддеревя по обратным рёбрам. Корневая вершина является точкой сочленения, если у неё не менее двух детей в дереве DFS. Не корневая вершина v является точкой сочленения, если существует хотя бы один её потомок u , из поддерева которого нет обратного ребра в предка v или более раннюю вершину. Сложность алгоритма поиска точек сочленения на основе обхода в глубину составляет $O(n + m)$, где n — количество вершин, а m — количество рёбер.

1.6 Алгоритм Дейкстры

Алгоритм Дейкстры находит оптимальные маршруты и их длину между одной конкретной вершиной (источником s) и вершиной t (всеми остальными вершинами) (ор)графа. Недостаток алгоритма: некорректно работает, если граф имеет рёбра (дуги) отрицательного веса. Сложность алгоритма Дейкстры составляет $O(p^2)$.

Принцип работы алгоритма: каждой вершине присваивается метка — текущее известное расстояние от источника. В начале источник получает постоянную метку 0, а все остальные — временные метки ∞ . На каждом шаге из всех временных меток выбирается вершина с наименьшим значением, её метка становится постоянной. Затем для всех непосредственных последователей этой вершины временные метки обновляются по правилу $d_{\text{нов}}(x_i) = \min\{d_{\text{стар}}(x_i), d(\tilde{x}) + \omega(\tilde{x}, x_i)\}$, где $\tilde{x}$ — текущая вершина (та, чья метка только что стала постоянной), а $\omega(\tilde{x}, x_i)$ — вес дуги от $\tilde{x}$ к x_i . Процесс повторяется, пока постоянная метка не будет присвоена каждой вершине графа; на выходе получаем вектор длин кратчайших путей от s . После этого кратчайший путь восстанавливается обратным ходом: начиная с t , на каждом шаге находится вершина x_i , удовлетворяющая $d(\tilde{x}) = d(x_i) + \omega(x_i, \tilde{x})$, и включается дуга $(x_i, \tilde{x})$ в путь; процесс продолжается до достижения источника s . Псевдокод алгоритма представлен на рисунке.

Вход: оргграф $G(V, E)$, заданный матрицей длин дуг $C : \text{array}[1..p, 1..p] \text{ of real}$ и списками смежности Γ ; s — исходная вершина графа.

Выход: вектор $T : \text{array}[1..p] \text{ of real}$ длин кратчайших путей от s .

```

for  $u \in V$  do
   $T[u] := C[s, u]$  { начальное приближение определяется матрицей }
   $X[u] := 0$  { все вершины не отмечены }
end for
for  $i$  from 1 to  $p$  do
   $m := \infty$  { поиск конца нового кратчайшего пути }
  for  $u \in V$  do
    if  $X[u] = 0 \ \& \ T[u] < m$  then
       $v := u, m := T[u]$  { вершина  $u$  заканчивает новый кратчайший путь из  $s$  }
    end if
  end for
  for  $u \in \Gamma(v)$  do
     $T[u] := \min(T[u], T[v] + C[v, u])$ 
    { пересчет оценки длины пути из  $s$  в  $u$  через  $v$  }
  end for
   $X[v] := 1$  { найден кратчайший путь из  $s$  в  $v$  }
end for

```

Рис. 2: Псевдокод построения дерева кратчайших путей.

1.7 Алгоритм Форда-Фалкерсона

Потоком в сети $S = (G; c)$, где $G = (V, E)$, называется такая функция $f : E \rightarrow \mathbb{R}_+$, приписывающая каждой дуге $e \in E$ неотрицательное число $f(e)$, называемое **потоком** по этой дуге e , для которой выполняются свойства:

1. для каждой дуги $e \in E$ верно $f(e) \leq c(e)$, т.е. поток по любой дуге не превышает пропускную способность этой дуги;
2. для каждой вершины $v \in V$, $v \neq s, t$, верно

$$D_f(v) = \sum_{e \in A(v)} f(e) - \sum_{e \in B(v)} f(e) = 0,$$

где $A(v)$ — дуги входящие в v , а $B(v)$ — дуги исходящие из v , т.е. поток, входящий в любую вершину, кроме источника и стока, без остатка выходит из нее (сохранение потока).

При этом неотрицательное число $p_f = \sum_{e \in B(s)} f(e)$ называется величиной потока f .

Алгоритм Форда-Фалкерсона находит максимальный поток в направленной сети с источником s и стоком t . Ключевую роль играют пропускные способности рёбер и остаточная сеть. Алгоритм работает, последовательно увеличивая поток вдоль увеличивающих путей в остаточном графе, пока такие пути существуют.

Алгоритм:

1. Для каждого ребра $(u \rightarrow v)$ устанавливаем поток $f(u, v) = 0$. Строим остаточную сеть G_f , где каждому ребру $(u \rightarrow v)$ с пропускной способностью $c(u, v)$ соответствуют прямое ребро с остаточной пропускной способностью $c_f(u, v) = c(u, v) - f(u, v)$ и обратное ребро с остаточной пропускной способностью $c_f(v, u) = f(u, v)$.

2. Находим любой путь от s до t в остаточной сети G_f , где все рёбра имеют положительную остаточную пропускную способность. Если такого пути нет — алгоритм завершён, текущий поток максимален.
3. Находим минимальную остаточную пропускную способность c_f на выбранном пути (это и есть возможное увеличение потока). Для каждого ребра $(u \rightarrow v)$ на пути: увеличиваем поток $f(u, v) += c_f$, уменьшаем остаточную пропускную способность $c_f(u, v) -= c_f$, увеличиваем остаточную пропускную способность обратного ребра $c_f(v, u) += c_f$.
4. Возвращаемся к шагу 2 и ищем новый увеличивающий путь в обновлённой остаточной сети.

В данной работе для поиска максимального потока используется реализация Эдмондса–Карпа, которая является частным случаем метода Форда–Фалкерсона. Отличие от описанного выше алгоритма заключается в способе выбора увеличивающего пути: на каждой итерации путь ищется с помощью поиска в ширину (BFS). Это гарантирует, что каждый найденный путь будет кратчайшим по числу рёбер в остаточной сети. Благодаря этому трудоёмкость алгоритма становится полиномиальной и составляет $O(nm^2)$, где n — число вершин, m — число рёбер (вместо $O(nmU)$ для исходного метода Форда–Фалкерсона, где U — максимальная пропускная способность сети).

1.8 Алгоритм поиска заданного потока минимальной стоимости

Рассматривается задача определения потока заданной величины ϑ от источника s к стоку t в сети $G = (S, U, \Omega, D)$, где каждая дуга $(x_i, x_j) \in U$ характеризуется пропускной способностью $c(x_i, x_j) \in \Omega$ и неотрицательной стоимостью $d(x_i, x_j) \in D$ пересылки единицы потока из x_i в x_j . Цель — минимизировать суммарную стоимость

$$\sum_{(x_i, x_j) \in U} d(x_i, x_j) \cdot \varphi(x_i, x_j)$$

при ограничениях:

1. $0 \leq \varphi(x_i, x_j) \leq c(x_i, x_j)$ для всех $(x_i, x_j) \in U$;
2. для источника s : $\sum_{x_i \in S} \varphi(s, x_i) - \sum_{x_j \in S} \varphi(x_j, s) = \vartheta$;
3. для стока t : $\sum_{x_i \in S} \varphi(t, x_i) - \sum_{x_j \in S} \varphi(x_j, t) = -\vartheta$;
4. для любой другой вершины $x_k \neq s, t$: $\sum_{x_i \in S} \varphi(x_k, x_i) - \sum_{x_j \in S} \varphi(x_j, x_k) = 0$ (сохранение потока).

Если $\vartheta > \varphi_{\max}$, где $\varphi_{\max}$ — величина максимального потока в сети, то решения не существует. В противном случае требуется найти поток минимальной стоимости.

В данной работе для нахождения потока минимальной стоимости заданной величины используется алгоритм последовательных кратчайших путей с потенциалами. Идея метода заключается в итеративном увеличении потока вдоль кратчайших по стоимости путей в остаточной сети. Для учёта возможных отрицательных стоимостей обратных дуг применяются приведённые стоимости с использованием потенциалов вершин, что позволяет использовать алгоритм Дейкстры для поиска кратчайшего пути на каждой итерации.

Построение остаточной сети (модифицированной относительно текущего потока φ): Пусть $G = (S, U, \Omega, D)$ — исходная сеть, а φ — некоторый поток. Модифицированная сеть $\tilde{G} = (\tilde{S}, \tilde{U}, \tilde{\Omega}, \tilde{D})$ строится следующим образом:

1. $\tilde{S} = S$ (множество вершин не изменяется);
2. $\tilde{U} = U \cup V$, где V — множество фиктивных дуг (обратных);
3. для каждой дуги $(x_i, x_j) \in U$, по которой проходит положительный поток ($\varphi(x_i, x_j) > 0$), добавляется обратная дуга (x_j, x_i) с параметрами:

$$\tilde{c}(x_j, x_i) = \varphi(x_i, x_j), \quad \tilde{d}(x_j, x_i) = -d(x_i, x_j);$$

4. для всех ненасыщенных дуг (включая те, по которым поток нулевой) полагается:

$$\tilde{c}(x_i, x_j) = c(x_i, x_j) - \varphi(x_i, x_j), \quad \tilde{d}(x_i, x_j) = d(x_i, x_j);$$

5. для насыщенных дуг ($\varphi(x_i, x_j) = c(x_i, x_j)$) полагается:

$$\tilde{c}(x_i, x_j) = 0, \quad \tilde{d}(x_i, x_j) = \infty.$$

Алгоритм:

1. Начальный поток полагается нулевым, потенциалы всех вершин равны нулю.
2. На каждой итерации в остаточной сети с помощью алгоритма Дейкстры ищется кратчайший путь из s в t по приведённым стоимостям

$$\tilde{d}_{\text{прив}}(u, v) = \tilde{d}(u, v) + \text{pot}(u) - \text{pot}(v),$$

которые неотрицательны при корректном обновлении потенциалов.

3. Если путь не найден, алгоритм завершается — достигнут максимальный поток.
4. Вдоль найденного пути определяется максимально возможное увеличение потока δ (минимальное из остаточных пропускных способностей дуг на пути и оставшегося требуемого потока ϑ — достигнутый поток).
5. Поток увеличивается на δ вдоль всех дуг пути (прямые дуги увеличиваются, обратные — уменьшаются). Соответственно обновляются остаточные пропускные способности в модифицированной сети.
6. Суммарная стоимость увеличивается на $\delta \cdot \sum \tilde{d}(u, v)$ по дугам пути (с учётом знака для обратных дуг).
7. Потенциалы вершин обновляются: $\text{pot}(v) \leftarrow \text{pot}(v) + \text{dist}(v)$ для всех достижимых вершин v .
8. Шаги 2–7 повторяются, пока не будет достигнут требуемый поток ϑ или не останется увеличивающих путей.

Сложность алгоритма составляет $O(\vartheta n^2)$ при использовании Дейкстры с матрицей смежности, где n — число вершин, ϑ — величина целевого потока.

1.9 Матричная теорема Кирхгофа

Пусть $G(V, E)$ — граф. Остовный подграф графа $G(V, E)$ — это подграф, содержащий все вершины. Остовный подграф, являющийся деревом, называется остовом.

Остов определяется множеством рёбер, поскольку вершины остова — это вершины исходного графа. Несвязный граф не имеет остова. Связный граф может иметь много остовов.

Определим матрицу Кирхгофа $B = B(G) = (\beta_{ij})_{n \times n}$ неориентированного графа G порядка n . Положим

$$B(G) = \begin{pmatrix} \deg v_1 & 0 & \cdots & 0 \\ 0 & \deg v_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \deg v_n \end{pmatrix} - A(G),$$

где $A(G)$ — матрица смежности графа G . Иными словами,

$$\beta_{ij} = \begin{cases} -1, & \text{если } i \neq j \text{ и } v_i \text{ смежна с } v_j, \\ 0, & \text{если } i \neq j \text{ и } v_i \text{ не смежна с } v_j, \\ \deg v_i, & \text{если } i = j. \end{cases}$$

Теорема Кирхгофа. Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа $B(G)$. Кроме того, все алгебраические дополнения матрицы Кирхгофа равны между собой.

Сложность вычисления числа остовных деревьев по матричной теореме Кирхгофа составляет $O(n^3)$, где n — число вершин графа.

1.10 Алгоритм Борувки

Алгоритм Борувки используется для нахождения кратчайшего остова (минимального остовного дерева) связного взвешенного графа. Сложность алгоритма Борувки составляет $O(m \log n)$, где n — количество вершин в графе, а m — количество рёбер. Пусть T — подграф графа G . Изначально T содержит все вершины G и не содержит рёбер. Далее выполняется следующий процесс:

1. Пока T не является деревом (что эквивалентно условию: число компонент связности $k = 1$), для каждой компоненты связности текущего леса T находится ребро минимального веса, которое соединяет вершину данной компоненты с вершиной, не принадлежащей этой компоненте.
2. В T добавляются все рёбра, которые оказались минимальными хотя бы для одной компоненты связности.

Полученный в результате граф T является минимальным остовным деревом исходного графа. Следует учитывать, что алгоритм может работать некорректно при наличии рёбер одинакового веса. Например, в полном графе из трёх вершин, где все рёбра имеют вес 1, в T могут быть добавлены все три ребра, что приведёт к циклу. Чтобы избежать этой проблемы, при выборе минимального ребра среди равных по весу следует выбирать ребро с наименьшим номером.

1.11 Код Прюфера

Код Прюфера устанавливает взаимно однозначное соответствие между деревьями с p вершинами и последовательностями длины $p - 2$ из чисел $1, \dots, p$ (с возможными повторениями). Это позволяет экономно хранить структуру дерева.

Построение кода (кодирование): Исходное дерево имеет вершины, занумерованные числами от 1 до p (произвольным образом). На каждом шаге из текущего дерева выбирается висячая вершина (степени 1) с наименьшим номером, в код записывается номер единственного соседа этой вершины, после чего выбранная вершина удаляется. Процесс повторяется $p - 2$ раза; последнее оставшееся ребро восстанавливается однозначно, поэтому его можно не хранить.

Восстановление дерева по коду (декодирование): Дан код $A[1..p - 2]$ (последовательность чисел от 1 до p). Множество ещё не использованных вершин B изначально содержит все номера $1, \dots, p$. Для i от 1 до $p - 2$ выбирается наименьшая вершина v из B , которая не встречается в оставшейся части кода $A[i], A[i + 1], \dots, A[p - 2]$, и добавляется ребро $(v, A[i])$, после чего v удаляется из B . По завершении цикла в B остаются две вершины, которые соединяются последним ребром.

В лабораторной работе при кодировании и декодировании необходимо сохранять не только номера вершин, но и веса рёбер, чтобы после декодирования остов имел те же веса. Сложность построения кода Прюфера (кодирование) составляет $O(n)$, где n — число вершин. Декодирование также выполняется за $O(n)$.

1.12 Алгоритм нахождения минимальной раскраски графа

Раскраской графа называется приписывание цветов (натуральных чисел) его вершинам такое, что никакие две смежные вершины не получают одинаковый цвет. Множество вершин одного цвета образует независимое множество (одноцветный класс). Наименьшее возможное число цветов есть хроматическое число $\chi(G)$, при этом выполняется $\chi(G) \leq 1 + \Delta(G)$, где $\Delta(G)$ — максимальная степень вершины.

В данной работе для построения допустимой раскраски используется улучшенный алгоритм последовательного раскрашивания. Его идея — сначала упорядочить вершины по невозрастанию их степеней. Затем алгоритм работает по цветам: на первом шаге все ещё не раскрашенные вершины просматриваются в указанном порядке, и если ни один сосед вершины ещё не покрашен в текущий цвет, вершина получает этот цвет и удаляется из дальнейшего рассмотрения. Когда все возможные вершины покрашены в текущий цвет, алгоритм переходит к следующему цвету и повторяет процесс. Так продолжается, пока все вершины не получают цвета. Полученная раскраска является допустимой, однако не обязательно минимальной; тем не менее эвристика с сортировкой по степени часто даёт результат, близкий к оптимальному. Сложность улучшенного последовательного алгоритма раскраски графа $O(n^2)$, где n — число вершин.

1.13 Эйлеров граф

Если граф имеет цикл (не обязательно простой), содержащий все рёбра графа, то такой цикл называется эйлеровым циклом, а граф называется эйлеровым. Если граф имеет цепь, содержащую все рёбра, то такая цепь называется эйлеровой цепью, а граф — полуйлеровым. Эйлеров цикл содержит каждое ребро ровно один раз, а вершины могут повторяться. Эйлеровым может быть только связный граф.

Справедлива теорема Эйлера: для связного нетривиального графа следующие утверждения эквивалентны:

1. Граф является эйлеровым.
2. Каждая вершина графа имеет чётную степень.
3. Множество рёбер графа можно разбить на простые циклы.

Для построения эйлерова цикла в эйлеровом графе используется алгоритм Флери. Он работает следующим образом.

1. Создаётся пустой стек для хранения вершин.
2. Выбирается произвольная вершина и помещается в стек.
3. Пока стек не пуст, выполняется:
 - v — вершина на вершине стека.
 - Если у вершины v нет смежных рёбер, то v извлекается из стека и добавляется в эйлеров цикл.
 - Иначе выбирается любая смежная вершина u , ребро (v, u) удаляется из графа, а вершина u помещается в стек.

Ключевая особенность алгоритма заключается в том, что при наличии выбора следует избегать рёбер, являющихся мостами, поскольку их удаление может привести к невозможности пройти оставшиеся рёбра. Ребро является мостом, если его удаление увеличивает количество компонент связности графа; проверка выполняется с помощью поиска в ширину (BFS) до и после временного удаления ребра. Если других рёбер нет, мост может быть пройден.

Ниже представлен псевдокод алгоритма:

Вход: эйлеров граф $G(V, E)$, заданный списками смежности ($\Gamma[v]$ — список вершин, смежных с вершиной v).

Выход: последовательность вершин эйлерова цикла.

```

 $S := \emptyset$  { стек для хранения вершин }
select  $v \in V$  { произвольная вершина }
 $v \rightarrow S$  { положить  $v$  в стек  $S$  }
while  $S \neq \emptyset$  do
   $v := \text{top } S$  {  $v$  — верхний элемент стека }
  if  $\Gamma[v] = \emptyset$  then
     $v \leftarrow S$ ; yield  $v$  { очередная вершина эйлерова цикла }
  else
    select  $u \in \Gamma[v]$  { взять первую вершину из списка смежности }
     $u \rightarrow S$  { положить  $u$  в стек }
     $\Gamma[v] := \Gamma[v] - u$ ;  $\Gamma[u] := \Gamma[u] - v$  { удалить ребро  $(v, u)$  }
  end if
end while

```

Рис. 3: Псевдокод алгоритма Флери.

Сложность проверки существования эйлерова цикла (проверка чётности степеней) составляет $O(n)$, где n — число вершин. Сложность построения эйлерова цикла алгоритмом Флери составляет $O(m^2)$, где m — количество рёбер.

1.14 Разрез графа

Разрезом связного графа называется множество рёбер, удаление которых делает граф несвязным. Любое разбиение множества вершин V на два непустых подмножества: $V_1 \neq \emptyset, V_2 \neq \emptyset, V_1 \cap V_2 = \emptyset, V_1 \cup V_2 = V$ определяет разрез $S = \{(v_1, v_2) \in E \mid v_1 \in V_1 \text{ \& } v_2 \in V_2\}$

1.15 Фундаментальная система разрезов

Пусть $T(V, E_T)$ — некоторый остов связного графа $G(V, E)$. Для каждого ребра остова $e \in E_T$ рассмотрим разрез S_e , определяемый следующим образом. Поскольку e является мостом в дереве T , его удаление разбивает множество вершин V на два непустых подмножества V_1 и V_2 ($V_1 \cup V_2 = V, V_1 \cap V_2 = \emptyset$). Включим в разрез S_e само ребро e и все хорды остова T (то есть рёбра графа G , не принадлежащие T), соединяющие вершины из V_1 с вершинами из V_2 . Иными словами, $S_e = E(V_1, V_2)$ — множество всех рёбер графа G между V_1 и V_2 , которое является простым разрезом. Совокупность разрезов $\{S_e\}_{e \in E_T}$ называется фундаментальной системой разрезов графа G относительно остова T . Количество разрезов в этой системе совпадает с числом рёбер остова и равно коциклическому рангу графа, обозначаемому $m^*(G)$. Сложность построения фундаментальной системы разрезов составляет $O(n(n + m))$, где n — число вершин, m — число рёбер. Для одного разреза — $O(n + m)$.

2 Особенности реализации

2.1 Структуры данных

Файл `common_structures.h` содержит описание структуры `Graph` для представления ориентированного графа в виде списков смежности, а также глобальные переменные для хранения текущего графа (матрица смежности), весовой матрицы, матрицы пропускных способностей и матрицы стоимостей. Управление этими матрицами осуществляется через функции, объявленные в этом же файле.

```
1  #include <iostream>
2  using namespace std;
3
4  struct Graph {
5      int N;
6      int** list;
7      int* size;
8
9      Graph(int n);
10     ~Graph();
11     void edge(int a, int b);
12     void print();
13 };
14
15 extern int** current_matrix;
16 extern int current_matrix_N;
17 extern bool current_matrix_oriented;
18
19 extern int** current_weight_matrix;
20 extern int current_weight_matrix_N;
21 extern bool weight_matrix_exist;
22
23 extern int** current_capacity_matrix;
24 extern int capacity_matrix_N;
25 extern bool capacity_matrix_exist;
26
27 extern int** current_cost_matrix;
28 extern int cost_matrix_N;
29 extern bool cost_matrix_exist;
30
31 void print_smezhn_matrix(int** matrix, int N);
32 void delete_matrix(int** M, int N);
33 void set_current_matrix(int** matrix, int N, bool oriented);
34 int** get_current_matrix(int& N, bool& oriented);
35
36 void set_weight_matrix(int** matrix, int N);
37 int** get_weight_matrix(int& N);
38 bool is_weight_matrix_exist();
39 void clear_weight_matrix();
40 void print_weight_matrix(int** weight, int N);
41
42 void set_capacity_matrix(int** matrix, int N);
43 int** get_capacity_matrix(int& N);
44 bool is_capacity_matrix_exist();
45 void clear_capacity_matrix();
46 void print_capacity_matrix(int** cap, int N);
47
48 void set_cost_matrix(int** matrix, int N);
49 int** get_cost_matrix(int& N);
50 bool is_cost_matrix_exist();
```

```

51 void clear_cost_matrix();
52 void print_cost_matrix(int** cost, int N);

```

Листинг 1: common_structures.h

2.2 Связный ациклический граф

Генерация ориентированного связного ациклического графа выполняется с использованием случайной перестановки вершин. Каждому ребру присваивается направление от вершины с меньшим порядковым номером в перестановке к вершине с большим номером, что гарантирует отсутствие циклов. Для обеспечения связности производится несколько попыток генерации (до 20), после чего граф проверяется на связность в неориентированном смысле.

Вход: int N — количество вершин графа

Выход: int** — матрица смежности размера $N \times N$, представляющая ориентированный связный ациклический граф

```

1  int** generate_directed_by_degrees(int N) {
2      int* out_degree = new int[N];
3
4      int max_deg = 0;
5      for (int i = 0; i < N; ++i) {
6          out_degree[i] = generate_random_degree();
7          if (out_degree[i] < 0) out_degree[i] = 0;
8          if (out_degree[i] > max_deg) max_deg = out_degree[i];
9      }
10
11     if (max_deg == 0) {
12         for (int i = 0; i < N; ++i) out_degree[i] = 1;
13         max_deg = 1;
14     }
15
16     double k = double(N - 1) / double(max_deg);
17
18     for (int i = 0; i < N; ++i) {
19         out_degree[i] = int(round(out_degree[i] * k));
20     }
21
22     cout << "Исходящие степени (после нормировки): ";
23     for (int i = 0; i < N; ++i) cout << out_degree[i] << " ";
24     cout << endl;
25
26     vector<int> order(N);
27     for (int i = 0; i < N; ++i) order[i] = i;
28     random_shuffle(order.begin(), order.end());
29
30     vector<int> position(N);
31     for (int i = 0; i < N; ++i) {
32         position[order[i]] = i;
33     }
34
35     int** adj = new int*[N];
36     for (int i = 0; i < N; ++i) {
37         adj[i] = new int[N];
38         for (int j = 0; j < N; ++j) adj[i][j] = 0;
39     }
40
41     for (int i = 0; i < N; ++i) {
42         int deg = out_degree[i];

```



```

43     vector<int> candidates;
44
45     for (int j = 0; j < N; ++j) {
46         if (j != i && position[j] > position[i]) {
47             candidates.push_back(j);
48         }
49     }
50
51     if (candidates.empty()) continue;
52
53     random_shuffle(candidates.begin(), candidates.end());
54     int edges_to_add = min(deg, (int)candidates.size());
55
56     for (int k2 = 0; k2 < edges_to_add; ++k2) {
57         adj[i][candidates[k2]] = 1;
58     }
59 }
60
61 delete[] out_degree;
62 return adj;
63 }

```

Листинг 2: Генерация графа (generate_graphs.cpp)

2.3 Распределение Чампернауна

Для генерации случайных целых чисел (степеней вершин, весов рёбер, пропускных способностей) используется непрерывное распределение Чампернауна с параметрами μ и α :

$$X = \mu + \frac{1}{\alpha} \ln \left(\tan \frac{\pi r}{2} \right), \quad r \in (0, 1).$$

Полученное значение округляется до ближайшего целого. Функция `generate_random_degree_champernaun` применяется как при создании графа, так и при генерации весовых матриц и матриц пропускных способностей.

Вход:

1. double μ — параметр сдвига
2. double α — параметр масштаба

Выход: int (для целочисленного) — сгенерированное случайное значение степени вершины

```

1  double generate_champernaun_continuous(double mu, double alpha) {
2      double r;
3      do {
4          r = (rand() + 0.5) / (RAND_MAX + 1.0);
5      } while (r <= 0.0 || r >= 1.0);
6      return mu + (1.0 / alpha) * log(tan(M_PI * r / 2.0));
7  }
8
9  int generate_random_degree_champernaun(double mu, double alpha) {
10     double x = generate_champernaun_continuous(mu, alpha);
11     int deg = (int)round(x);
12     if (deg < 0) deg = 0;
13     return deg;
14 }

```

Листинг 3: Генерация по Чампернауну (generate_graphs.cpp)

2.4 Метод Шимбелла

Метод Шимбелла позволяет найти кратчайшие пути между всеми парами вершин с ограничением на максимальное число рёбер. Реализовано обобщённое умножение матриц, где операция «сложения» заменена на минимум (или максимум), а «умножения» — на сложение весов. Итеративное возведение в степень (до *max_edges*) даёт искомые характеристики.

Вход:

1. `int** A, int** B` — матрицы весов (размер $N \times N$)
2. `int N` — размер матриц
3. `bool max_min` — `false`: поиск минимума, `true`: поиск максимума

Выход: `int**` — матрица C размера $N \times N$, где $C[i][j]$ — минимальная (или максимальная) сумма весов.

```
1  int** shimbell_multiply(int** A, int** B, int N, bool max_min) {
2      int** C = new int*[N];
3      for (int i = 0; i < N; i++) {
4          C[i] = new int[N];
5          for (int j = 0; j < N; j++) {
6              bool found = false;
7              int best = (max_min) ? -INF : INF;
8              for (int k = 0; k < N; k++) {
9                  if (A[i][k] == INF || B[k][j] == INF) continue;
10                 if (A[i][k] == 0 || B[k][j] == 0) continue;
11                 int sum = A[i][k] + B[k][j];
12                 if (!found) { best = sum; found = true; }
13                 else if (!max_min && sum < best) best = sum;
14                 else if (max_min && sum > best) best = sum;
15             }
16             C[i][j] = found ? best : INF;
17         }
18     }
19     return C;
20 }
```

Листинг 4: Умножение матриц по Шимбеллу (shimbell.cpp)

2.5 Точки сочленения графа

Поиск точек сочленения в неориентированном графе, выполняется с помощью обхода в глубину (DFS). В процессе обхода для каждой вершины v вычисляются время входа $disc[v]$ и минимальная достижимая метка $low[v]$ (наименьшее значение $disc$ среди вершин, достижимых из поддерева v по обратным рёбрам). Условие $low[u] \geq disc[v]$ для ребёнка u в DFS-дереве определяет, является ли v точкой сочленения (для корня условие иное: наличие двух и более сыновей). Алгоритм работает за время $O(n^2)$, где n — число вершин, что обусловлено просмотром всей матрицы смежности.

Вход:

1. `int** matrix` — матрица смежности ($N \times N$, неориентированный граф)
2. `int N` — количество вершин

Выход: список вершин, являющихся точками сочленения (выводится на экран)

```

1  void find_articulation_points_matrix(int** matrix, int N)
2  {
3      if (!matrix || N <= 0) {
4          cout << "Граф пуст или некорректен\n";
5          return;
6      }
7
8      bool* visited = new bool[N];
9      int* disc = new int[N];
10     int* low = new int[N];
11     bool* isAP = new bool[N];
12
13     for (int i = 0; i < N; ++i) {
14         visited[i] = false;
15         disc[i] = 0;
16         low[i] = 0;
17         isAP[i] = false;
18     }
19
20     int timer = 0;
21
22     ap_dfs(0, -1, matrix, N, visited, disc, low, timer, isAP);
23
24     cout << "Точки сочленения: ";
25     bool any = false;
26     for (int i = 0; i < N; ++i) {
27         if (isAP[i]) {
28             cout << i << " ";
29             any = true;
30         }
31     }
32     if (!any) {
33         cout << "(нет)";
34     }
35     cout << "\n";
36
37     delete[] visited;
38     delete[] disc;
39     delete[] low;
40     delete[] isAP;
41 }

```

Листинг 5: Поиск точек сочленения (articulation.cpp)

2.6 Алгоритм Дейкстры

Классический алгоритм Дейкстры реализован с проверкой на наличие отрицательных весов (в этом случае выводится сообщение об ошибке). Расстояния хранятся в массиве *dist*, родители — в *parent*. На каждом шаге выбирается непосещённая вершина с минимальным расстоянием, затем релаксируются все исходящие рёбра.

Вход:

1. `int** W` — весовая матрица ($N \times N$, 0 означает отсутствие ребра)
2. `int N` — количество вершин
3. `int start` — стартовая вершина
4. `int* dist`, `int* parent` — выходные массивы

5. long long& iters — счётчик итераций (по ссылке)

Выход: bool — true, если алгоритм выполнен успешно (нет отрицательных весов); иначе false.

dist[N] — массив кратчайших расстояний, parent[N] — массив предков, iters — количество итераций.

```
1  bool dijkstra_shortest(int** W, int N, int start, int* dist, int* parent,
2      long long& iters) {
3      if (has_negative_weights(W, N)) {
4          return false;
5      }
6      bool* used = new bool[N];
7      for (int i = 0; i < N; ++i) {
8          dist[i] = INF; parent[i] = -1; used[i] = false;
9      }
10     dist[start] = 0;
11     for (int step = 0; step < N; ++step) {
12         int v = -1;
13         for (int i = 0; i < N; ++i)
14             if (!used[i] && (v == -1 || dist[i] < dist[v])) v = i;
15         if (v == -1 || dist[v] == INF) break;
16         used[v] = true;
17         for (int u = 0; u < N; ++u) {
18             if (W[v][u] == 0) continue;
19             if (dist[v] + W[v][u] < dist[u]) {
20                 dist[u] = dist[v] + W[v][u];
21                 parent[u] = v;
22             }
23         }
24     }
25     delete[] used;
26     return true;
27 }
```

Листинг 6: Дейкстра (shortest_paths.cpp)

2.7 Алгоритм Форда-Фалкерсона (Эдмондс-Карп)

Поиск максимального потока в ориентированной сети выполняется с помощью BFS для нахождения увеличивающего пути в остаточной сети. Алгоритм Эдмондса-Карпа гарантирует полиномиальное время $O(nm^2)$.

Вход:

1. int** capacity — матрица пропускных способностей ($N \times N$, целые неотрицательные)
2. int N — количество вершин
3. int s, int t — источник и сток

Выход: int — величина максимального потока из s в t

```
1  int max_flow_edmonds_karp(int** capacity, int N, int s, int t) {
2      if (!capacity || N <= 0) return 0;
3      if (s < 0 || s >= N || t < 0 || t >= N) return 0;
4
5      int** residual = new int*[N];
6      for (int i = 0; i < N; ++i) {
7          residual[i] = new int[N];
```

```

8     for (int j = 0; j < N; ++j) {
9         residual[i][j] = capacity[i][j];
10    }
11 }
12
13 int* parent = new int[N];
14 int max_flow = 0;
15
16 while (true) {
17     int flow = bfs_for_flow(residual, N, s, t, parent);
18     if (flow == 0) break;
19     max_flow += flow;
20
21     int cur = t;
22     while (cur != s) {
23         int prev = parent[cur];
24         residual[prev][cur] -= flow;
25         residual[cur][prev] += flow;
26         cur = prev;
27     }
28 }
29
30 delete[] parent;
31 delete_matrix(residual, N);
32 return max_flow;
33 }
34

```

Листинг 7: Максимальный поток (flows.cpp)

2.8 Алгоритм поиска заданного потока минимальной стоимости

Для нахождения потока заданной величины с минимальной суммарной стоимостью используется метод последовательного дополнения вдоль кратчайших путей в приведённой сети. Для поиска кратчайших путей используется ранее реализованный алгоритм Дейкстры. На каждом шаге находится путь минимальной стоимости в остаточной сети, по которому пропускается максимально возможный поток.

Вход:

1. `int** capacity` — матрица пропускных способностей ($N \times N$)
2. `int** cost` — матрица стоимостей ($N \times N$)
3. `int N` — количество вершин
4. `int s, int t` — источник и сток
5. `int required_flow` — требуемая величина потока
6. `int& achieved_flow` — выходной параметр
7. `int** flow` — матрица для записи найденного потока

Выход: `long long` — минимальная суммарная стоимость достигнутого потока.

Через параметры: `achieved_flow` — реально достигнутая величина потока ($\leq \text{required_flow}$), `flow[N][N]` — матрица потока на каждом ребре.

```

1  long long min_cost_flow_with_dijkstra( int** capacity,
2  int** cost, int N, int s, int t, int required_flow,
3  int& achieved_flow, int** flow
4  ) {
5      achieved_flow = 0;
6      if (!capacity || !cost || N <= 0) return 0;
7      if (s < 0 || s >= N || t < 0 || t >= N) return 0;
8      if (required_flow <= 0) return 0;
9
10     int** cap = new int*[N];
11     for (int i = 0; i < N; ++i) {
12         cap[i] = new int[N];
13         for (int j = 0; j < N; ++j) {
14             cap[i][j] = capacity[i][j];
15         }
16     }
17
18     int** c = new int*[N];
19     for (int i = 0; i < N; ++i) {
20         c[i] = new int[N];
21         for (int j = 0; j < N; ++j) {
22             c[i][j] = cost[i][j];
23         }
24     }
25
26     for (int i = 0; i < N; ++i) {
27         for (int j = 0; j < N; ++j) {
28             flow[i][j] = 0;
29         }
30     }
31
32     vector<int> potential(N, 0);
33     vector<int> dist(N);
34     vector<int> parent(N);
35
36     long long total_cost = 0;
37
38     while (achieved_flow < required_flow) {
39         dijkstra_reduced(cap, c, N, s, potential, dist, parent);
40         if (dist[t] == INF_LOCAL) break;
41
42         for (int i = 0; i < N; ++i) {
43             if (dist[i] < INF_LOCAL) {
44                 potential[i] += dist[i];
45             }
46         }
47
48         int add_flow = required_flow - achieved_flow;
49         int v = t;
50         while (v != s) {
51             int u = parent[v];
52             if (u == -1) { add_flow = 0; break; }
53             if (cap[u][v] < add_flow) add_flow = cap[u][v];
54             v = u;
55         }
56         if (add_flow <= 0) break;
57
58         v = t;
59         while (v != s) {
60             int u = parent[v];

```

```

61         cap[u][v] -= add_flow;
62         cap[v][u] += add_flow;
63
64         flow[u][v] += add_flow;
65         flow[v][u] -= add_flow;
66
67         total_cost += 1LL * add_flow * cost[u][v];
68
69         v = u;
70     }
71
72     achieved_flow += add_flow;
73 }
74
75 for (int i = 0; i < N; ++i) {
76     delete[] cap[i];
77     delete[] c[i];
78 }
79 delete[] cap;
80 delete[] c;
81
82 return total_cost;
83 }

```

Листинг 8: Поток минимальной стоимости (flows.cpp)

2.9 Матричная теорема Кирхгофа

Число остовных деревьев неориентированного графа вычисляется как любой главный минор матрицы Кирхгофа. Матрица Кирхгофа L определяется: $B_{ii} = \deg(i)$, $B_{ij} = -1$ при наличии ребра между i и j . Определитель матрицы $(N - 1) \times (N - 1)$ находится методом Гаусса с выбором главного элемента и использованием типа `long double` для повышения точности.

Вход:

1. `int** adj` — матрица смежности неориентированного графа ($N \times N$, 1 — ребро, 0 — нет)
2. `int N` — количество вершин

Выход: `long long` — число остовных деревьев графа (округляется до ближайшего целого от определителя)

```

1  static long double determinant_ld(vector<vector<long double>>& A, int n) {
2      long double det = 1.0L;
3      for (int i = 0; i < n; ++i) {
4          int pivot = i;
5          long double maxv = fabs1(A[i][i]);
6          for (int r = i+1; r < n; ++r) {
7              long double val = fabs1(A[r][i]);
8              if (val > maxv) { maxv = val; pivot = r; }
9          }
10         if (fabs1(maxv) < 1e-18L) {
11             return 0.0L;
12         }
13         if (pivot != i) {
14             swap(A[pivot], A[i]);
15             det = -det;
16         }

```

```

17     long double diag = A[i][i];
18     det *= diag;
19     for (int r = i+1; r < n; ++r) {
20         long double factor = A[r][i] a diag;
21         if (fabsl(factor) < 1e-18L) continue;
22         for (int c = i; c < n; ++c) {
23             A[r][c] -= factor * A[i][c];
24         }
25     }
26 }
27 return det;
28 }

```

Листинг 9: Подсчёт остовных деревьев (kirchhoff.cpp)

2.10 Алгоритм Борувки

Построение минимального остовного дерева (MST) во взвешенном неориентированном графе выполняется алгоритмом Борувки. На каждом этапе для каждой компоненты связности выбирается инцидентное ребро минимального веса, после чего компоненты объединяются. Алгоритм продолжается, пока не останется одна компонента.

Вход:

1. `int** adj` — матрица смежности неориентированного графа ($N \times N$)
2. `int** weight` — матрица весов рёбер ($N \times N$, 0 — ребро отсутствует)
3. `int N` — количество вершин
4. `int& total_weight` — выходной параметр для суммарного веса MST

Выход: `int**` — матрица смежности минимального остовного дерева (размер $N \times N$).
`total_weight` — суммарный вес рёбер в построенном MST.

```

1  int** boruvka_mst(int** adj, int** weight, int N, int& total_weight) {
2      total_weight = 0;
3      if (!adj || !weight || N <= 0) return nullptr;
4
5      int** mst = new int*[N];
6      for (int i=0; i<N; ++i){
7          mst[i] = new int[N];
8          for (int j=0; j<N; ++j) mst[i][j] = 0;
9      }
10
11     DSU dsu(N);
12     int components = N;
13     total_weight = 0;
14
15     while (components > 1) {
16         const int INF_W = numeric_limits<int>::max();
17         vector<int> best_u(N, -1);
18         vector<int> best_v(N, -1);
19         vector<int> best_w(N, INF_W);
20         vector<int> best_id(N, numeric_limits<int>::max());
21
22         for (int u = 0; u < N; ++u) {
23             for (int v = 0; v < N; ++v) {
24                 if (u == v) continue;
25                 if (adj[u][v] == 0 && adj[v][u] == 0) continue;

```



```

26     int w = weight[u][v];
27     if (w == 0) continue;
28     int cu = dsu.find(u);
29     int cv = dsu.find(v);
30     if (cu == cv) continue;
31     int eid = u * N + v;
32     if (w < best_w[cu] || (w == best_w[cu] && eid < best_id[cu])) {
33         best_w[cu] = w;
34         best_u[cu] = u;
35         best_v[cu] = v;
36         best_id[cu] = eid;
37     }
38 }
39 }
40
41
42 for (int i = 0; i < N; ++i) {
43     if (best_u[i] == -1) continue;
44     int u = best_u[i];
45     int v = best_v[i];
46     int cu = dsu.find(u);
47     int cv = dsu.find(v);
48     if (cu == cv) continue;
49
50     if (dsu.unite(cu, cv)) {
51         mst[u][v] = 1;
52         mst[v][u] = 1;
53         total_weight += weight[u][v];
54         components--;
55     }
56 }
57 }
58
59 clear_last_mst();
60 last_mst_N = N;
61 last_mst = new int*[N];
62 for (int i=0; i<N; ++i){
63     last_mst[i] = new int[N];
64     for (int j=0; j<N; ++j) last_mst[i][j] = mst[i][j];
65 }
66
67 return mst;
68 }

```

Листинг 10: Алгоритм Борувки (boruvka.cpp)

2.11 Код Прюфера

Для кодирования дерева (MST) в код Прюфера с сохранением весов рёбер реализованы две функции: `encode_prufer_with_weights` и `decode_prufer_with_weights`. При кодировании на каждом шаге удаляется лист с наименьшим номером, а номер смежной вершины заносится в код. Вес удаляемого ребра сохраняется в отдельный список. При декодировании восстанавливается дерево и соответствующие веса.

Кодирование (`encode_prufer_with_weights`):

Вход:

1. `int** adj` — матрица смежности дерева (MST)
2. `int** weight` — матрица весов рёбер

3. int N — количество вершин

Выход: bool — успешность операции.

vector<int>& code_nodes — код Прюфера (длина N-2), vector<int>& code_weights — веса удаляемых рёбер, int& last_u, int& last_v — вершины последнего ребра.

```
1 bool encode_prufer_with_weights(  
2 int** adj,  
3 int** weight,  
4 int N,  
5 vector<int>& code_nodes,  
6 vector<int>& code_weights,  
7 int& last_u,  
8 int& last_v  
9 ) {  
10     if (!adj || !weight || N <= 0) return false;  
11  
12     int** tmp_adj = new int* [N];  
13     for (int i = 0; i < N; ++i) {  
14         tmp_adj[i] = new int[N];  
15         for (int j = 0; j < N; ++j) {  
16             tmp_adj[i][j] = adj[i][j];  
17         }  
18     }  
19  
20     vector<int> deg(N, 0);  
21     for (int i = 0; i < N; ++i) {  
22         for (int j = 0; j < N; ++j) {  
23             if (tmp_adj[i][j] == 1) deg[i]++;  
24         }  
25     }  
26  
27     priority_queue<int, vector<int>, greater<int>> leaves;  
28     for (int i = 0; i < N; ++i) {  
29         if (deg[i] == 1) leaves.push(i);  
30     }  
31  
32     code_nodes.clear();  
33     code_weights.clear();  
34     code_nodes.reserve((N > 2) ? (N - 2) : 0);  
35     code_weights.reserve((N > 1) ? (N - 1) : 0);  
36  
37     for (int step = 0; step < N - 2; ++step) {  
38         int v = -1;  
39         while (!leaves.empty()) {  
40             int x = leaves.top();  
41             leaves.pop();  
42             if (deg[x] == 1) { v = x; break; }  
43         }  
44         if (v == -1) {  
45             for (int i = 0; i < N; ++i) delete[] tmp_adj[i];  
46             delete[] tmp_adj;  
47             return false;  
48         }  
49  
50         int u = -1;  
51         for (int j = 0; j < N; ++j) {  
52             if (tmp_adj[v][j] == 1) {  
53                 u = j;  
54                 break;  
55             }  
56         }
```

```

56     }
57     if (u == -1) {
58         for (int i = 0; i < N; ++i) delete[] tmp_adj[i];
59         delete[] tmp_adj;
60         return false;
61     }
62
63     code_nodes.push_back(u);
64     code_weights.push_back(weight[v][u]);
65
66     tmp_adj[v][u] = tmp_adj[u][v] = 0;
67     deg[v]--;
68     deg[u]--;
69
70     if (deg[u] == 1) {
71         leaves.push(u);
72     }
73 }
74
75 last_u = -1;
76 last_v = -1;
77 for (int i = 0; i < N; ++i) {
78     if (deg[i] == 1) {
79         if (last_u == -1) last_u = i;
80         else last_v = i;
81     }
82 }
83 if (last_u == -1 || last_v == -1) {
84     for (int i = 0; i < N; ++i) delete[] tmp_adj[i];
85     delete[] tmp_adj;
86     return false;
87 }
88
89 code_weights.push_back(weight[last_u][last_v]);
90
91 for (int i = 0; i < N; ++i) delete[] tmp_adj[i];
92 delete[] tmp_adj;
93 return true;
94 }

```

Листинг 11: Кодирование Прюфера с весами (prufer.cpp)

Декодирование (decode_prufer_with_weights):

Вход:

1. `vector<int>& code_nodes` — код Прюфера (вершины)
2. `vector<int>& code_weights` — соответствующие веса
3. `int N` — количество вершин

Выход: `int**` — матрица смежности восстановленного дерева ($N \times N$), `int**& decoded_weights` — матрица весов.

```

1  int** decode_prufer_with_weights(
2  const vector<int>& code_nodes,
3  const vector<int>& code_weights,
4  int N,
5  int**& decoded_weights
6  ) {
7      decoded_weights = nullptr;

```

```

8   if (N <= 0) return nullptr;
9   if ((int)code_nodes.size() != N - 2) return nullptr;
10  if ((int)code_weights.size() != N - 1) return nullptr;
11
12  int** adj = new int* [N];
13  for (int i = 0; i < N; ++i) {
14      adj[i] = new int[N];
15      for (int j = 0; j < N; ++j) adj[i][j] = 0;
16  }
17
18  decoded_weights = new int* [N];
19  for (int i = 0; i < N; ++i) {
20      decoded_weights[i] = new int[N];
21      for (int j = 0; j < N; ++j) decoded_weights[i][j] = 0;
22  }
23
24  vector<int> deg(N, 1);
25  for (int x : code_nodes) {
26      if (x >= 0 && x < N) deg[x]++;
27  }
28
29  priority_queue<int, vector<int>, greater<int>> leaves;
30  for (int i = 0; i < N; ++i) {
31      if (deg[i] == 1) leaves.push(i);
32  }
33
34  for (int i = 0; i < N - 2; ++i) {
35      int u = code_nodes[i];
36      int w = code_weights[i];
37
38      int v = -1;
39      while (!leaves.empty()) {
40          int x = leaves.top();
41          leaves.pop();
42          if (deg[x] == 1) { v = x; break; }
43      }
44      if (v == -1) {
45          for (int r = 0; r < N; ++r) delete[] adj[r];
46          delete[] adj;
47          for (int r = 0; r < N; ++r) delete[] decoded_weights[r];
48          delete[] decoded_weights;
49          decoded_weights = nullptr;
50          return nullptr;
51      }
52
53      adj[v][u] = adj[u][v] = 1;
54      decoded_weights[v][u] = decoded_weights[u][v] = w;
55
56      deg[v]--;
57      deg[u]--;
58
59      if (deg[u] == 1) {
60          leaves.push(u);
61      }
62  }
63
64  int a = -1, b = -1;
65  for (int i = 0; i < N; ++i) {
66      if (deg[i] == 1) {
67          if (a == -1) a = i;

```

```

68         else b = i;
69     }
70 }
71 if (a != -1 && b != -1) {
72     int last_weight = code_weights.back();
73     adj[a][b] = adj[b][a] = 1;
74     decoded_weights[a][b] = decoded_weights[b][a] = last_weight;
75 } else {
76     for (int r = 0; r < N; ++r) delete[] adj[r];
77     delete[] adj;
78     for (int r = 0; r < N; ++r) delete[] decoded_weights[r];
79     delete[] decoded_weights;
80     decoded_weights = nullptr;
81     return nullptr;
82 }
83
84 return adj;
85 }

```

Листинг 12: Декодирование Прюфера с весами (prufer.cpp)

2.12 Алгоритм нахождения минимальной раскраски графа

Для раскраски вершин неориентированного графа используется улучшенный алгоритм последовательного раскрашивания. Вершины сортируются по убыванию степени, затем последовательно раскрашиваются в минимальный возможный цвет. Выбор цвета осуществляется с учётом уже окрашенных соседей.

Вход:

1. `int** adj` — матрица смежности неориентированного графа ($N \times N$)
2. `int N` — количество вершин
3. `int& colors_used` — выходной параметр

Выход: `int*` — массив цветов вершин (длина `N`, цвета от 1 до `colors_used`).

Через параметр `colors_used` — количество использованных цветов (одноцветных классов).

```

1  int* dsatur_coloring(int** adj, int N, int& colors_used) {
2      std::vector<int> degree(N, 0);
3      for (int i = 0; i < N; ++i) {
4          int deg = 0;
5          for (int j = 0; j < N; ++j) {
6              if (adj[i][j] != 0) ++deg;
7          }
8          degree[i] = deg;
9      }
10
11     std::vector<int> vertices(N);
12     for (int i = 0; i < N; ++i) vertices[i] = i;
13     std::sort(vertices.begin(), vertices.end(),
14         [&](int a, int b) { return degree[a] > degree[b]; });
15
16     std::vector<int> color(N, 0);
17     int uncolored = N;
18     int c = 1;
19     colors_used = 0;
20
21     while (uncolored > 0) {

```

```

22     for (int idx = 0; idx < N; ++idx) {
23         int v = vertices[idx];
24         if (color[v] != 0) continue;
25
26         bool conflict = false;
27         for (int u = 0; u < N; ++u) {
28             if (adj[v][u] != 0 && color[u] == c) {
29                 conflict = true;
30                 break;
31             }
32         }
33         if (!conflict) {
34             color[v] = c;
35             --uncolored;
36         }
37     }
38     if (c > colors_used) colors_used = c;
39     ++c;
40 }
41
42 int* result = new int[N];
43 for (int i = 0; i < N; ++i) result[i] = color[i];
44 return result;
45 }

```

Листинг 13: Раскраска графа (coloring.cpp)

2.13 Эйлеров граф

Проверка графа на эйлеровость заключается в вычислении чётности степеней всех вершин. Если граф не эйлеров, выполняется модификация: вершины с нечётной степенью попарно соединяются (добавляются или удаляются рёбра) для достижения чётности, затем строится эйлеров цикл.

Проверка (is_eulerian):

Вход:

1. `int** adj` — матрица смежности неориентированного графа ($N \times N$)
2. `int N` — количество вершин

Выход: `bool` — `true`, если все вершины имеют чётную степень (граф эйлеров).

```

1  bool is_eulerian(int** adj, int N) {
2      for (int i = 0; i < N; ++i) {
3          int deg = 0;
4          for (int j = 0; j < N; ++j) deg += (adj[i][j] != 0);
5          if (deg % 2 != 0) return false;
6      }
7      return true;
8  }

```

Листинг 14: Проверка на эйлеровость (euler.cpp)

Модификация (make_eulerian):

Вход:

1. `int** adj` — исходная матрица смежности

2. int N — количество вершин

Выход: vector<vector<int>& mult — обновлённая матрица смежности с чётными степенями.

```
1 void make_eulerian(int** adj, int N, vector<vector<int>>& eul) {
2     eul.assign(N, vector<int>(N, 0));
3     for (int i = 0; i < N; ++i)
4         for (int j = 0; j < N; ++j)
5         eul[i][j] = adj[i][j];
6
7     vector<int> deg(N, 0);
8     for (int i = 0; i < N; ++i)
9         for (int j = 0; j < N; ++j)
10        if (adj[i][j] != 0) deg[i]++;
11
12    vector<int> odds;
13    for (int i = 0; i < N; ++i)
14        if (deg[i] % 2 == 1)
15            odds.push_back(i);
16
17    if (odds.empty()) {
18        cout << "Граф уже эйлеров.\n";
19        return;
20    }
21
22    sort(odds.begin(), odds.end(), [&](int a, int b) {
23        return deg[a] < deg[b];
24    });
25
26    cout << "Вершины с нечётной степенью (отсортированы по степени): ";
27    for (int x : odds) cout << x << " ";
28    cout << endl;
29
30    vector<bool> used(odds.size(), false);
31    int remaining = odds.size();
32
33    while (remaining > 0) {
34        int i = 0;
35        while (i < (int)odds.size() && used[i]) ++i;
36        if (i == (int)odds.size()) break;
37
38        int u = odds[i];
39        int j;
40        for (int k = i + 1; k < (int)odds.size(); ++k) {
41            if (used[k]) continue;
42            int v = odds[k];
43            if (deg[u] == 1 || deg[v] == 1) {
44                if (eul[u][v] == 0) {
45                    j = k;
46                    break;
47                }
48            } else {
49                j = k;
50                break;
51            }
52        }
53
54        int v = odds[j];
55        if (eul[u][v] > 0) {
56            eul[u][v]--;
```

```

57     eul[v][u]--;
58     cout << "Удалено ребро " << u << " - " << v << endl;
59 } else {
60     eul[u][v]++;
61     eul[v][u]++;
62     cout << "Добавлено ребро " << u << " - " << v << endl;
63 }
64 used[i] = used[j] = true;
65 remaining -= 2;
66 }
67 }

```

Листинг 15: Модификация графа в эйлеров (euler.cpp)

Построение цикла (find_eulerian_cycle):

Вход:

1. `const vector<vector<int>>& eul` — матрица кратностей эйлерова графа
2. `int N` — количество вершин
3. `int start` — начальная вершина

Выход: `vector<int>` — последовательность вершин эйлерова цикла.

```

1  vector<int> find_eulerian_cycle(const vector<vector<int>>& eul, int N, int
    start) {
2      vector<vector<int>> m = eul;
3      vector<int> cycle;
4      stack<int> st;
5      st.push(start);
6
7      while (!st.empty()) {
8          int v = st.top();
9          bool found = false;
10         for (int u = 0; u < N; ++u) {
11             if (m[v][u] > 0) {
12                 m[v][u]--;
13                 m[u][v]--;
14                 st.push(u);
15                 found = true;
16                 break;
17             }
18         }
19         if (!found) {
20             cycle.push_back(v);
21             st.pop();
22         }
23     }
24
25     reverse(cycle.begin(), cycle.end());
26     return cycle;
27 }

```

Листинг 16: Эйлеров цикл (euler.cpp)

2.14 Разрез графа

Разрез — это множество рёбер, удаление которых увеличивает число компонент связности. В программе разрез задаётся как набор пар вершин (u, v) (рёбра соединяющие

эти вершины). Фундаментальный разрез для ребра e минимального остовного дерева строится как множество всех рёбер исходного графа, соединяющих две компоненты, полученные удалением e из минимального остовного дерева (множество хорд).

В файле `cuts.h` определён тип `Cut` как вектор пар целых чисел, а также объявлены функции для работы с фундаментальными разрезами.

```

1  #include <vector>
2  #include <utility>
3
4  using Cut = std::vector<std::pair<int,int>>;
5
6  std::vector<Cut> get_fundamental_cuts(int** adj, int** mst, int N);
7  void print_cut(const Cut& cut, const char* name);
8  Cut symmetric_difference(const Cut& a, const Cut& b);
9  Cut symmetric_difference_multiple(const std::vector<Cut>& cuts);

```

Листинг 17: cuts.h

2.15 Фундаментальная система разрезов

Совокупность фундаментальных разрезов для всех рёбер MST образует фундаментальную систему разрезов. Над выбранными разрезами может быть выполнена операция симметрической разности, которая также является разрезом. Реализованы функции `symmetric_difference` и `symmetric_difference_multiple`.

Построение фундаментального разреза для ребра минимального остовного дерева (`get_fundamental_cut_for_edge`):

Вход:

1. `int** adj` — матрица смежности исходного графа
2. `int** mst` — матрица смежности остовного дерева
3. `int N` — количество вершин
4. `int u_edge, int v_edge` — ребро остовного дерева

Выход: `Cut` (тип `vector<pair<int,int>`) — полученный фундаментальный разрез.

```

1  Cut get_fundamental_cut_for_edge(int** adj, int** mst, int N, int u_edge,
2  int v_edge) {
3      vector<bool> comp1;
4      get_component(mst, N, u_edge, u_edge, v_edge, comp1);
5      Cut cut;
6      if (u_edge < v_edge) cut.push_back({u_edge, v_edge});
7      else cut.push_back({v_edge, u_edge});
8
9      for (int i = 0; i < N; ++i) {
10         for (int j = i+1; j < N; ++j) {
11             if (adj[i][j] == 0) continue;
12             if (comp1[i] != comp1[j]) {
13                 cut.push_back({i, j});
14             }
15         }
16     }
17     sort(cut.begin(), cut.end());
18     cut.erase(unique(cut.begin(), cut.end()), cut.end());
19     return cut;

```

Листинг 18: Фундаментальный разрез (cuts.cpp)

Симметрическая разность двух разрезов (symmetric_difference):

Вход:

1. const Cut& a, const Cut& b — два разреза

Выход: Cut — разрез, являющийся симметрической разностью фундаментальных разрезов.

```
1  Cut symmetric_difference(const Cut& a, const Cut& b) {
2      set<pair<int,int>> sa(a.begin(), a.end());
3      set<pair<int,int>> sb(b.begin(), b.end());
4      Cut res;
5      for (auto& p : sa) {
6          if (sb.find(p) == sb.end()) res.push_back(p);
7      }
8      for (auto& p : sb) {
9          if (sa.find(p) == sa.end()) res.push_back(p);
10     }
11     sort(res.begin(), res.end());
12     return res;
13 }
```

Листинг 19: Фундаментальный разрез (cuts.cpp)

Симметрическая разность нескольких разрезов (symmetric_difference_multiple):

Вход:

1. const vector<Cut>& cuts — список разрезов

Выход: Cut — результирующий разрез, после последовательного применения симметрической разности ко всем разрезам из списка.

```
1  Cut symmetric_difference_multiple(const vector<Cut>& cuts) {
2      if (cuts.empty()) return Cut();
3      Cut res = cuts[0];
4      for (size_t i = 1; i < cuts.size(); ++i) {
5          res = symmetric_difference(res, cuts[i]);
6      }
7      return res;
8  }
```

Листинг 20: Фундаментальный разрез (cuts.cpp)

2.16 Меню программы (main.cpp)

В главном файле main.cpp реализовано текстовое меню, позволяющее пользователю последовательно вызывать все реализованные алгоритмы. После генерации графа все данные сохраняются в глобальных переменных, что позволяет выполнять над ними различные операции без повторной генерации. Меню поддерживает следующие действия (см. листинг ниже).

```
1 #include <iostream>
2 #include <cstdlib>
3 #include <ctime>
4 #include <string>
5 #include <vector>
```

```

6 #include <climits>
7 #include "common_structures.h"
8 #include "generate_graphs.h"
9 #include "metrics.h"
10 #include "shimbell.h"
11 #include "routes.h"
12 #include "articulation.h"
13 #include "shortest_paths.h"
14 #include "flows.h"
15 #include "boruvka.h"
16 #include "coloring.h"
17 #include "kirchhoff.h"
18 #include "prufer.h"
19 #include "euler.h"
20 #include "cuts.h"
21
22 using namespace std;
23
24 void print_menu() {
25     cout << "\nМЕНЮ\n";
26     cout << "1. Сгенерировать ориентированный связный ациклический граф (по степеням Чампернауна)\n";
27     cout << "2. Сгенерировать неориентированный граф из текущего ориентированного (симметризация матриц)\n";
28     cout << "3. Сгенерировать весовую матрицу для текущего графа (положить атрибут веса)\n";
29     cout << "4. Показать текущий граф и метрики (эксцентриситеты, центр, диаметр)\n";
30     cout << "5. Показать сохранённую весовую матрицу\n";
31     cout << "6. Метод Шимбелла (минимум/максимум путей с ограничением по числу рёбер)\n";
32     cout << "7. Подсчитать количество маршрутов между двумя вершинами\n";
33     cout << "8. Найти точки сочленения в текущем графе (только для неориентированного)\n";
34     cout << "9. Найти кратчайший путь между двумя вершинами (Классический алгоритм Дейкстры)\n";
35     cout << "10. Сгенерировать матрицы пропускных способностей и стоимостей для текущего графа\n";
36     cout << "11. Показать матрицы пропускных способностей и стоимостей\n";
37     cout << "12. Найти максимальный поток (алгоритм Эдмондса-Карпа)\n";
38     cout << "13. Найти поток минимальной стоимости объёма [2a3 * max]\n";
39     cout << "14. Число остовных деревьев (теорема Кирхгофа) для текущего неориентированного графа\n";
40     cout << "15. Построить минимальное остовное дерево (алгоритм Борувки) и сохранить его как последний MST\n";
41     cout << "16. Код Прюфера: закодировать последний MST и декодировать обратно (с сохранением весов)\n";
42     cout << "17. Минимальная раскраска графа (на исходном графе или на последнем MST)\n";
43     cout << "18. Проверить граф на эйлеровость, при необходимости модифицировать и построить эйлеров цикл\n";
44     cout << "19. Фундаментальная система разрезов последнего MST и симметричная разность выбранных разрезов\n";
45     cout << "0. Выход\n";
46     cout << "Ваш выбор: ";
47 }
48
49 aa Вспомогательная функция для ввода целого числа с проверкой диапазона и возможностью выхода (-1)

```

```

50 int input_int(const string& prompt, int min_val, int max_val, bool
    allow_cancel = true) {
51     int val;
52     while (true) {
53         cout << prompt;
54         if (!(cin >> val)) {
55             cin.clear();
56             cin.ignore(10000, '\n');
57             cout << "Ошибка ввода. Введите целое число.\n";
58             continue;
59         }
60         if (allow_cancel && val == -1) {
61             cout << "Отмена операции.\n";
62             return -1;
63         }
64         if (val >= min_val && val <= max_val) {
65             return val;
66         }
67         cout << "Значение должно быть в диапазоне [" << min_val << ", " <<
max_val << "]. Повторите.\n";
68     }
69 }
70
71 int main() {
72     srand((unsigned)time(nullptr));
73     int choice;
74     int N = 0;
75
76     do {
77         print_menu();
78         cin >> choice;
79         if (cin.fail()) {
80             cin.clear();
81             cin.ignore(10000, '\n');
82             cout << "Некорректный ввод. Повторите.\n";
83             continue;
84         }
85
86         switch (choice) {
87             case 1: {
88                 aa Ввод N
89                 int n = input_int("Введите количество вершин (или -1 для отмены): ",
1, INT_MAX, true);
90                 if (n == -1) break;
91                 N = n;
92
93                 clear_weight_matrix();
94                 clear_capacity_matrix();
95                 clear_cost_matrix();
96
97                 int** adj = nullptr;
98                 for (int attempt = 0; attempt < 20; ++attempt) {
99                     adj = generate_directed_by_degrees(N);
100
101                     bool* vis = new bool[N];
102                     for (int i = 0; i < N; ++i) vis[i] = false;
103                     int* q = new int[N]; int h = 0, t = 0;
104                     q[t++] = 0; vis[0] = true;
105                     while (h < t) {
106                         int v = q[h++];

```

```

107         for (int u = 0; u < N; ++u) {
108             if ((adj[v][u] == 1 || adj[u][v] == 1) && !vis[u]) {
109                 vis[u] = true; q[t++] = u;
110             }
111         }
112     }
113     bool ok = true;
114     for (int i = 0; i < N; ++i) if (!vis[i]) { ok = false; break; }
115     delete[] vis; delete[] q;
116     if (ok) break;
117     delete_matrix(adj, N); adj = nullptr;
118 }
119 if (!adj) {
120     cout << "Не удалось сгенерировать связный ориентированный граф\n";
121     break;
122 }
123 set_current_matrix(adj, N, true);
124 delete_matrix(adj, N);
125 cout << "Ориентированный связный ациклический граф сгенерирован и со
хранён как текущая матрица.\n";
126 break;
127 }
128 case 2: {
129     int curN; bool oriented;
130     int** cur = get_current_matrix(curN, oriented);
131     if (!cur) {
132         cout << "Нет текущего графа. Сначала сгенерируйте ориентированный
граф в пункте 1.\n";
133         break;
134     }
135     if (!oriented) {
136         cout << "Текущий граф уже неориентированный.\n";
137         break;
138     }
139
140     int** und = new int* [curN];
141     for (int i = 0; i < curN; ++i) {
142         und[i] = new int[curN];
143         for (int j = 0; j < curN; ++j) und[i][j] = 0;
144     }
145     for (int i = 0; i < curN; ++i) {
146         for (int j = 0; j < curN; ++j) {
147             if (cur[i][j] == 1 || cur[j][i] == 1) {
148                 und[i][j] = 1;
149                 und[j][i] = 1;
150             }
151         }
152     }
153     set_current_matrix(und, curN, false);
154     delete_matrix(und, curN);
155
156     if (is_weight_matrix_exist()) {
157         int wn; int** W = get_weight_matrix(wn);
158         if (wn == curN) {
159             int** Wsym = new int* [wn];
160             for (int i = 0; i < wn; ++i) {
161                 Wsym[i] = new int[wn];
162                 for (int j = 0; j < wn; ++j) Wsym[i][j] = 0;
163             }
164             for (int i = 0; i < wn; ++i) {

```

```

165         for (int j = i; j < wn; ++j) {
166             int a = W[i][j];
167             int b = W[j][i];
168             int val = 0;
169             if (a != 0 && b != 0) val = a;
170             else if (a != 0) val = a;
171             else if (b != 0) val = b;
172             else val = 0;
173             Wsym[i][j] = val;
174             Wsym[j][i] = val;
175         }
176     }
177     set_weight_matrix(Wsym, wn);
178     delete_matrix(Wsym, wn);
179 } else {
180     cout << "Размер сохранённой весовой матрицы не совпадает с текущ
им графом 6 весовая матрица не симметризована.\n";
181 }
182 }
183     cout << "Преобразование выполнено: текущая матрица теперь неориентир
ованная. Весовая матрица симметризована (если была сохранена и совпадала
по размеру).\n";
184     break;
185 }
186 case 3: {
187     int curN; bool oriented;
188     int** cur = get_current_matrix(curN, oriented);
189     if (!cur) {
190         cout << "Нет текущего графа\n";
191         break;
192     }
193     int sign = input_int("Выберите тип весов: 1-положительные, 2-отрицат
ельные, 3-смешанные (или -1 для отмены): ", 1, 3, true);
194     if (sign == -1) break;
195
196     int** W = new int* [curN];
197     for (int i = 0; i < curN; ++i) {
198         W[i] = new int[curN];
199         for (int j = 0; j < curN; ++j) W[i][j] = 0;
200     }
201
202     double weight_mu = 8.0;
203     double weight_alpha = 0.9;
204
205     for (int i = 0; i < curN; ++i) {
206         for (int j = 0; j < curN; ++j) {
207             bool has = oriented ? (cur[i][j] == 1) : (i < j && (cur[i][j] ==
1 || cur[j][i] == 1));
208             if (!has) continue;
209             int w = 0;
210             while (w == 0) w = generate_random_degree_champernaun(weight_mu,
weight_alpha);
211             if (sign == 2) w = -w;
212             else if (sign == 3 && rand() % 2 == 0) w = -w;
213             W[i][j] = w;
214             if (!oriented && i < j) W[j][i] = w;
215         }
216     }
217     set_weight_matrix(W, curN);
218     cout << "Весовая матрица сгенерирована и сохранена.\n";

```

```

219         delete_matrix(W, curN);
220         break;
221     }
222     case 4: {
223         int curN; bool oriented;
224         int** cur = get_current_matrix(curN, oriented);
225         if (!cur) {
226             cout << "Нет текущего графа\n";
227             break;
228         }
229         cout << (oriented ? "Текущий граф: ориентированный\n" : "Текущий гра
ф: неориентированный\n");
230         print_smezhn_matrix(cur, curN);
231
232         if (!oriented) {
233             Graph* g = new Graph(curN);
234             for (int i = 0; i < curN; ++i) {
235                 for (int j = i + 1; j < curN; ++j) {
236                     if (cur[i][j] == 1 || cur[j][i] == 1) g->edge(i, j);
237                 }
238             }
239             int* eccs = all_eccentricities(g);
240             cout << "Экцентриситеты:\n";
241             for (int i = 0; i < curN; ++i) cout << "e(" << i << ")=" << eccs[i
] << "\n";
242             find_center(eccs, curN);
243             find_diametr(eccs, curN);
244             delete[] eccs;
245             delete g;
246         } else {
247             int* eccs = eccentricities_oriented(cur, curN);
248             print_eccentricities_oriented(eccs, curN);
249             find_center_oriented(eccs, curN);
250             find_diametr_oriented(eccs, curN);
251             delete[] eccs;
252         }
253         break;
254     }
255     case 5: {
256         if (!is_weight_matrix_exist()) {
257             cout << "Весовая матрица не сохранена. Сгенерируйте её в пункте
3.\n";
258             break;
259         }
260         int wn; int** W = get_weight_matrix(wn);
261         cout << "Сохранённая весовая матрица:\n";
262         print_weight_matrix(W, wn);
263         break;
264     }
265     case 6: {
266         int curN; bool oriented;
267         int** cur = get_current_matrix(curN, oriented);
268         if (!cur) {
269             cout << "Нет текущего графа\n";
270             break;
271         }
272         if (!is_weight_matrix_exist()) {
273             cout << "Нет сохранённой весовой матрицы (сгенерируйте в пункте 3)
\n";
274             break;

```

```

275     }
276     int Nw; int** W = get_weight_matrix(Nw);
277     int maxe = input_int("Введите максимальное число рёбер в пути (0 - т
олько диагональ, -1 для отмены): ", 0, INT_MAX, true);
278     if (maxe == -1) break;
279     int t = input_int("1 - минимальные пути, 2 - максимальные, 3 - оба (
или -1 для отмены): ", 1, 3, true);
280     if (t == -1) break;
281     if (t == 1 || t == 3) {
282         int** minp = shimbell_all_paths(W, Nw, maxe, false);
283         print_matrix_with_inf(minp, Nw, "Минимальные пути");
284         delete_matrix(minp, Nw);
285     }
286     if (t == 2 || t == 3) {
287         int** maxp = shimbell_all_paths(W, Nw, maxe, true);
288         print_matrix_with_inf(maxp, Nw, "Максимальные пути");
289         delete_matrix(maxp, Nw);
290     }
291     break;
292 }
293 case 7: {
294     int curN; bool oriented;
295     int** cur = get_current_matrix(curN, oriented);
296     if (!cur) {
297         cout << "Нет текущего графа\n";
298         break;
299     }
300
301     int s = input_int("Стартовая вершина (или -1 для отмены): ", 0, curN
- 1, true);
302     if (s == -1) break;
303     int e = input_int("Конечная вершина (или -1 для отмены): ", 0, curN
- 1, true);
304     if (e == -1) break;
305
306     int cnt = count_routes_matrix(cur, curN, oriented, s, e);
307     if (cnt < 0) {
308         cout << "Ошибка: некорректные номера вершин\n";
309     } else if (cnt == 0) {
310         cout << "Маршрут из вершины " << s << " в вершину " << e << " отсу
тствует\n";
311     } else {
312         cout << "Количество маршрутов: " << cnt << "\n";
313     }
314     break;
315 }
316 case 8: {
317     int curN; bool oriented;
318     int** cur = get_current_matrix(curN, oriented);
319     if (!cur) {
320         cout << "Нет текущего графа\n";
321         break;
322     }
323     if (oriented) {
324         cout << "Точки сочленения определяются для неориентированных графо
в.\n";
325         cout << "Сначала преобразуйте граф в неориентированный (пункт 2).\n
n";
326         break;
327     }

```



```

328     cout << "Текущий неориентированный граф (матрица смежности):\n";
329     print_smezhn_matrix(cur, curN);
330     find_articulation_points_matrix(cur, curN);
331     break;
332 }
333 case 9: {
334     if (!is_weight_matrix_exist()) {
335         cout << "Нет весовой матрицы. Сгенерируйте её в пункте 3.\n";
336         break;
337     }
338     int Nw;
339     int** W = get_weight_matrix(Nw);
340
341     int s = input_int("Стартовая вершина (или -1 для отмены): ", 0, Nw -
1, true);
342     if (s == -1) break;
343     int e = input_int("Конечная вершина (или -1 для отмены): ", 0, Nw -
1, true);
344     if (e == -1) break;
345
346     int* dist = new int[Nw];
347     int* parent = new int[Nw];
348     long long iters = 0;
349
350     bool ok = dijkstra_shortest(W, Nw, s, dist, parent, iters);
351     if (!ok) {
352         delete[] dist;
353         delete[] parent;
354         break;
355     }
356
357     cout << "Вектор расстояний (Дейкстра):\n";
358     for (int i = 0; i < Nw; ++i) {
359         cout << "dist[" << i << "] = ";
360         if (dist[i] >= INF) cout << "INF";
361         else cout << dist[i];
362         cout << "\n";
363     }
364
365     cout << "Кратчайший путь " << s << " -> " << e << ": ";
366     if (dist[e] >= INF) {
367         cout << "пути нет\n";
368     } else {
369         print_path_from_parents(s, e, parent);
370         cout << "\nДлина пути: " << dist[e] << "\n";
371     }
372     cout << "Количество итераций: " << iters << "\n";
373
374     delete[] dist;
375     delete[] parent;
376     break;
377 }
378 case 10: {
379     generate_capacity_and_cost_for_current_graph();
380     break;
381 }
382 case 11: {
383     if (!is_capacity_matrix_exist()) {
384         cout << "Матрица пропускных способностей не сохранена. Сгенерируйт
е её в пункте 10.\n";

```

```

385     } else {
386         int nc; int** C = get_capacity_matrix(nc);
387         print_capacity_matrix(C, nc);
388     }
389     if (!is_cost_matrix_exist()) {
390         cout << "Матрица стоимостей не сохранена. Сгенерируйте её в пункте
10.\n";
391     } else {
392         int nw; int** Wc = get_cost_matrix(nw);
393         print_cost_matrix(Wc, nw);
394     }
395     break;
396 }
397 case 12: {
398     if (!is_capacity_matrix_exist()) {
399         cout << "Нет матрицы пропускных способностей. Сгенерируйте её в пу
нкте 10.\n";
400         break;
401     }
402     int Ncap; int** C = get_capacity_matrix(Ncap);
403     int s = input_int("Источник (s) (или -1 для отмены): ", 0, Ncap - 1,
true);
404     if (s == -1) break;
405     int t = input_int("Сток (t) (или -1 для отмены): ", 0, Ncap - 1,
true);
406     if (t == -1) break;
407
408     int maxflow = max_flow_edmonds_karp(C, Ncap, s, t);
409     cout << "Максимальный поток из " << s << " в " << t << " = " <<
maxflow << "\n";
410     break;
411 }
412 case 13: {
413     if (!is_capacity_matrix_exist() || !is_cost_matrix_exist()) {
414         cout << "Нет матриц пропускных способностей иили стоимостей. Сген
ерируйте их в пункте 10.\n";
415         break;
416     }
417     int Ncap, Ncost;
418     int** C = get_capacity_matrix(Ncap);
419     int** Wc = get_cost_matrix(Ncost);
420     if (Ncap != Ncost) {
421         cout << "Размеры матриц пропускных способностей и стоимостей не со
впадают.\n";
422         break;
423     }
424     int s = input_int("Источник (s) (или -1 для отмены): ", 0, Ncap - 1,
true);
425     if (s == -1) break;
426     int t = input_int("Сток (t) (или -1 для отмены): ", 0, Ncap - 1,
true);
427     if (t == -1) break;
428
429     int maxflow = max_flow_edmonds_karp(C, Ncap, s, t);
430     cout << "Максимальный поток из " << s << " в " << t << " = " <<
maxflow << "\n";
431     if (maxflow == 0) {
432         cout << "Максимальный поток равен 0, поток минимальной стоимости н
е имеет смысла.\n";
433         break;

```

```

434     }
435
436     int required_flow = (2 * maxflow) a 3;
437     if (required_flow <= 0) required_flow = 1;
438     cout << "Требуемый объём потока для минимальной стоимости: " <<
required_flow << " ( [2a3 * max] )\n";
439
440     int achieved_flow = 0;
441     int** flow = new int* [Ncap];
442     for (int i = 0; i < Ncap; ++i) {
443         flow[i] = new int[Ncap];
444         for (int j = 0; j < Ncap; ++j) flow[i][j] = 0;
445     }
446
447     long long cost = min_cost_flow_with_dijkstra(C, Wc, Ncap, s, t,
required_flow, achieved_flow, flow);
448     cout << "Достигнутый поток: " << achieved_flow << "\n";
449     cout << "Минимальная стоимость этого потока: " << cost << "\n";
450
451     vector<vector<int>> paths;
452     vector<int> path_flow;
453     vector<long long> path_cost;
454     decompose_flow_into_paths(flow, Ncap, s, t, paths, path_flow,
path_cost, Wc);
455
456     cout << "\nНайдено маршрутов: " << paths.size() << "\n";
457     int sumFlows = 0;
458     long long sumCosts = 0;
459     for (size_t i = 0; i < paths.size(); ++i) {
460         sumFlows += path_flow[i];
461         sumCosts += path_cost[i];
462     }
463     cout << "Суммарный поток по маршрутам: " << sumFlows << "\n";
464     cout << "Суммарная стоимость по маршрутам: " << sumCosts << "\n";
465
466     cout << "\nИспользованные маршруты:\n";
467     if (paths.empty()) {
468         cout << "(нет использованных маршрутов)\n";
469     } else {
470         for (size_t i = 0; i < paths.size(); ++i) {
471             for (size_t j = 0; j < paths[i].size(); ++j) {
472                 cout << paths[i][j];
473                 if (j + 1 < paths[i].size()) cout << " -> ";
474             }
475             long long flow_val = path_flow[i];
476             long long unit_cost = (flow_val == 0 ? 0 : path_cost[i] a
flow_val);
477             cout << " : " << flow_val << " * " << unit_cost << " = " <<
path_cost[i] << "\n";
478         }
479     }
480
481     for (int i = 0; i < Ncap; ++i) delete[] flow[i];
482     delete[] flow;
483     break;
484 }
485 case 14: {
486     int curN; bool oriented;
487     int** cur = get_current_matrix(curN, oriented);
488     if (!cur) {

```

```

489         cout << "Нет текущего графа. Сгенерируйте его в пункте 1.\n";
490         break;
491     }
492     if (oriented) {
493         cout << "Теорема Кирхгофа применяется к неориентированным графам.
Сначала преобразуйте (пункт 2).\n";
494         break;
495     }
496     long long trees = count_spanning_trees(cur, curN);
497     if (trees < 0) {
498         cout << "Ошибка при подсчёте\n";
499     } else {
500         cout << "Число остовных деревьев (по теореме Кирхгофа): " << trees
<< "\n";
501     }
502     break;
503 }
504 case 15: {
505     int curN; bool oriented;
506     int** cur = get_current_matrix(curN, oriented);
507     if (!cur) {
508         cout << "Нет текущего графа\n";
509         break;
510     }
511     if (oriented) {
512         cout << "Для Борувки нужен неориентированный граф. Сначала преобра
зуйте (пункт 2).\n";
513         break;
514     }
515     if (!is_weight_matrix_exist()) {
516         cout << "Нет весовой матрицы. Сгенерируйте её в пункте 3.\n";
517         break;
518     }
519     int wn; int** W = get_weight_matrix(wn);
520     if (wn != curN) {
521         cout << "Размер весовой матрицы не совпадает с текущим графом\n";
522         break;
523     }
524
525     int total_weight = 0;
526     int** mst = boruvka_mst(cur, W, curN, total_weight);
527     if (!mst) {
528         cout << "Не удалось построить MST (возможно граф несвязен)\n";
529         break;
530     }
531
532     cout << "MST (алгоритм Борувки) построено. Суммарный вес = " <<
total_weight << "\n";
533     cout << "Матрица смежности MST:\n";
534     print_smezhn_matrix(mst, curN);
535
536     cout << "Весовая матрица MST:\n";
537     for (int i = 0; i < curN; ++i) {
538         for (int j = 0; j < curN; ++j) {
539             if (mst[i][j] == 1)
540                 cout << W[i][j] << " ";
541             else
542                 cout << "0 ";
543         }
544         cout << "\n";

```

```

545     }
546
547     for (int i = 0; i < curN; ++i) delete[] mst[i];
548     delete[] mst;
549     break;
550 }
551 case 16: {
552     int mstN;
553     int** mst = get_last_mst(mstN);
554     if (!mst) {
555         cout << "Последний MST отсутствует. Сначала постройте MST (пункт
15).\n";
556         break;
557     }
558     if (!is_weight_matrix_exist()) {
559         cout << "Нет весовой матрицы. Сгенерируйте её в пункте 3.\n";
560         break;
561     }
562     int wn;
563     int** W = get_weight_matrix(wn);
564     if (wn != mstN) {
565         cout << "Размер весовой матрицы не совпадает с MST\n";
566         break;
567     }
568
569     cout << "Кодирование MST в код Прюфера (с сохранением весов)...\n";
570
571     vector<int> code_nodes;
572     vector<int> code_weights;
573     int last_u = -1, last_v = -1;
574
575     bool ok = encode_prufer_with_weights(mst, W, mstN, code_nodes,
code_weights, last_u, last_v);
576     if (!ok) {
577         cout << "Ошибка при кодировании Прюфера\n";
578         break;
579     }
580
581     cout << "Код Прюфера (вершины): ";
582     for (size_t i = 0; i < code_nodes.size(); ++i)
583         cout << code_nodes[i] << " ";
584     cout << "\nСоответствующие веса удаляемых рёбер: ";
585     for (size_t i = 0; i < code_weights.size(); ++i)
586         cout << code_weights[i] << " ";
587     cout << "\nПоследнее ребро: " << last_u << " - " << last_v
<< " (вес " << code_weights.back() << ")\n";
588
589     cout << "\nДекодирование кода Прюфера обратно в дерево...\n";
590
591     int** decoded_weights = nullptr;
592     int** decoded = decode_prufer_with_weights(code_nodes, code_weights,
mstN, decoded_weights);
593     if (!decoded) {
594         cout << "Ошибка при декодировании Прюфера\n";
595         break;
596     }
597 }
598
599     cout << "\nВосстановленное дерево (матрица смежности):\n";
600     print_smezhn_matrix(decoded, mstN);
601

```

```

602     cout << "\nВосстановленная весовая матрица:\n";
603     for (int i = 0; i < mstN; ++i) {
604         for (int j = 0; j < mstN; ++j) {
605             cout << decoded_weights[i][j] << " ";
606         }
607         cout << "\n";
608     }
609
610     for (int i = 0; i < mstN; ++i) {
611         delete[] decoded[i];
612         delete[] decoded_weights[i];
613     }
614     delete[] decoded;
615     delete[] decoded_weights;
616     break;
617 }
618 case 17: {
619     int opt = input_int("Выберите граф для раскраски: 1 - текущий граф,
620 2 - последний MST (если есть) (или -1 для отмены): ", 1, 2, true);
621     if (opt == -1) break;
622     int** target = nullptr;
623     int TN = 0;
624     if (opt == 1) {
625         int curN; bool oriented;
626         int** cur = get_current_matrix(curN, oriented);
627         if (!cur) {
628             cout << "Нет текущего графа\n";
629             break;
630         }
631         if (oriented) {
632             cout << "Раскраска реализована для неориентированных графов. Сна
633 чала преобразуйте (пункт 2).\n";
634             break;
635         }
636         target = cur;
637         TN = curN;
638     } else {
639         int mstN;
640         int** mst = get_last_mst(mstN);
641         if (!mst) {
642             cout << "Последний MST отсутствует. Сначала постройте MST (пункт
643 15).\n";
644             break;
645         }
646         target = mst;
647         TN = mstN;
648     }
649
650     int colors_used = 0;
651     int* colors = dsatur_coloring(target, TN, colors_used);
652     if (!colors) {
653         cout << "Ошибка при раскраске\n";
654         break;
655     }
656
657     cout << "Раскраска выполнена. Количество одноцветных классов: " <<
colors_used << "\n";
658     for (int i = 0; i < TN; ++i) {
659         cout << "Вершина " << i << " : цвет " << colors[i] << "\n";
660     }

```

```

658     delete[] colors;
659     break;
660 }
661 case 18: {
662     int curN; bool oriented;
663     int** cur = get_current_matrix(curN, oriented);
664     if (!cur) { cout << "Нет текущего графа...\n"; break; }
665     if (curN <= 1){
666         cout << "Граф не может быть эйлеровым\n";
667         continue;
668     }
669
670     if (is_eulerian(cur, curN)) {
671         cout << "Граф уже эйлеров.\n";
672         vector<vector<int>> mult;
673         make_eulerian(cur, curN, mult);
674         vector<int> cycle = find_eulerian_cycle(mult, curN);
675         if (cycle.empty()) cout << "Не удалось построить эйлеров цикл.\n";
676         else {
677             cout << "Эйлеров цикл: ";
678             for (size_t i = 0; i < cycle.size(); ++i) {
679                 cout << cycle[i];
680                 if (i+1 < cycle.size()) cout << " -> ";
681             }
682             cout << endl;
683         }
684     } else {
685         if (curN == 2){
686             cout << "Нельзя модифицировать данный граф в эйлеров.\n";
687             continue;
688         }
689         cout << "Граф не эйлеров. Модификация...\n";
690         vector<vector<int>> mult;
691         make_eulerian(cur, curN, mult);
692         cout << "Модифицированный граф:\n";
693         for (int i = 0; i < curN; ++i) {
694             for (int j = 0; j < curN; ++j) cout << mult[i][j] << " ";
695             cout << endl;
696         }
697         vector<int> cycle = find_eulerian_cycle(mult, curN);
698         if (cycle.empty()) cout << "Не удалось построить эйлеров цикл.\n";
699         else {
700             cout << "Эйлеров цикл в модифицированном графе: ";
701             for (size_t i = 0; i < cycle.size(); ++i) {
702                 cout << cycle[i];
703                 if (i+1 < cycle.size()) cout << " -> ";
704             }
705             cout << endl;
706         }
707     }
708     break;
709 }
710 case 19: {
711     int mstN;
712     int** mst = get_last_mst(mstN);
713     if (!mst) {
714         cout << "Последний MST отсутствует. Сначала постройте MST (пункт
15).\n";
715         break;
716     }

```

```

717     int curN; bool oriented;
718     int** cur = get_current_matrix(curN, oriented);
719     if (!cur) {
720         cout << "Нет текущего графа. Необходим неориентированный граф для
построения разрезов.\n";
721         break;
722     }
723     if (oriented) {
724         cout << "Разрезы строятся для неориентированного графа. Преобразуй
те граф (пункт 2).\n";
725         break;
726     }
727     if (curN != mstN) {
728         cout << "Размер текущего графа и MST не совпадают. Сначала построй
те MST для текущего графа.\n";
729         break;
730     }
731
732     vector<Cut> fundamental = get_fundamental_cuts(cur, mst, mstN);
733     if (fundamental.empty()) {
734         cout << "Не удалось получить фундаментальные разрезы (возможно MST
пусто).\n";
735         break;
736     }
737     cout << "Фундаментальная система разрезов (всего " << fundamental.
size() << " разрезов):\n";
738     for (size_t i = 0; i < fundamental.size(); ++i) {
739         string name = "Разрез " + to_string(i+1);
740         print_cut(fundamental[i], name.c_str());
741     }
742
743     cout << "\nВведите номера разрезов (через Enter) для получения их си
мметрической разности.\n";
744     cout << "Для завершения ввода введите -1.\n";
745     vector<int> indices;
746     while (true) {
747         int idx = input_int("Номер разреза (или -1 для завершения): ", 1,
(int)fundamental.size(), true);
748         if (idx == -1) break;
749         indices.push_back(idx-1);
750     }
751     if (indices.empty()) {
752         cout << "Не выбрано ни одного разреза.\n";
753         break;
754     }
755     vector<Cut> selected;
756     for (int i : indices) selected.push_back(fundamental[i]);
757     Cut result = symmetric_difference_multiple(selected);
758
759     // Вывод результата с проверкой на пустоту
760     if (result.empty()) {
761         cout << "Результирующий разрез: Пустой набор рёбер\n";
762     } else {
763         print_cut(result, "Результирующий разрез");
764     }
765     break;
766 }
767 case 0:
768     cout << "Выход\n";
769     break;

```



```

770         default:
771             cout << "Неверный выбор\n";
772     }
773 } while (choice != 0);
774
775 if (current_matrix) {
776     for (int i = 0; i < current_matrix_N; ++i) delete[] current_matrix[i];
777     delete[] current_matrix;
778     current_matrix = nullptr;
779 }
780 clear_weight_matrix();
781 clear_capacity_matrix();
782 clear_cost_matrix();
783 clear_last_mst();
784 return 0;
785 }

```

Листинг 21: main.cpp

3 Результаты работы

После запуска программы выводится меню с 19 пунктами (рис. 4).

```
МЕНЮ
1. Сгенерировать ориентированный связный ациклический граф (по степеням Чампернауна)
2. Сгенерировать неориентированный граф из текущего ориентированного (симметризация матриц)
3. Сгенерировать весовую матрицу для текущего графа (положит/отриц/смеш)
4. Показать текущий граф и метрики (эксцентриситеты, центр, диаметр)
5. Показать сохранённую весовую матрицу
6. Метод Шимбелла (минимум/максимум путей с ограничением по числу рёбер)
7. Подсчитать количество маршрутов между двумя вершинами
8. Найти точки сочленения в текущем графе (только для неориентированного)
9. Найти кратчайший путь между двумя вершинами (Классический алгоритм Дейкстры)
10. Сгенерировать матрицы пропускных способностей и стоимостей для текущего графа
11. Показать матрицы пропускных способностей и стоимостей
12. Найти максимальный поток (алгоритм Эдмондса-Карпа)
13. Найти поток минимальной стоимости объёма  $[2/3 * \max]$ 
14. Число остовных деревьев (теорема Кирхгофа) для текущего неориентированного графа
15. Построить минимальное остовное дерево (алгоритм Борувки) и сохранить его как последний MST
16. Код Прифера: закодировать последний MST и декодировать обратно (с сохранением весов)
17. Минимальная раскраска графа (на исходном графе или на последнем MST)
18. Проверить граф на эйлеровость, при необходимости модифицировать и построить эйлеров цикл
19. Фундаментальная система разрезов последнего MST и симметрическая разность выбранных разрезов
0. Выход
Ваш выбор: █
```

Рис. 4: Вывод меню программы

При вводе 1, происходит генерация ориентированного связного графа по введённым пользователем количеству вершин (рис. 5).

```
Ваш выбор: 1
Введите количество вершин (или -1 для отмены): 8
Исходящие степени (после нормировки): 2 2 0 7 2 2 5 2
Ориентированный связный ациклический граф сгенерирован и сохранён как текущая матрица.
```

Рис. 5: Генерация графа

На рис. 6 представлена генерация неориентированного графа из текущего ориентированного.

```
Ваш выбор: 2
Преобразование выполнено: текущая матрица теперь неориентированная. Весовая матрица симметризована (если была сохранена и совпадала по размеру).
```

Рис. 6: Генерация неориентированного графа

На рис. 7 представлена генерация весовой матрицы с настройкой весов: положительные, отрицательные и смешанные.

```
Ваш выбор: 3
Выберите тип весов: 1-положительные, 2-отрицательные, 3-смешанные (или -1 для отмены): 1
Весовая матрица сгенерирована и сохранена.
```

Рис. 7: Генерация весовой матрицы

На рис. 8 показан вывод матрицы смежности текущего графа и его метрик.

```
Ваш выбор: 4
Текущий граф: неориентированный
0 0 0 1 0 0 1 1
0 0 0 1 1 0 0 0
0 0 0 0 1 0 1 0
1 1 0 0 1 1 1 0
0 1 1 1 0 0 1 1
0 0 0 1 0 0 1 0
1 0 1 1 1 1 0 0
1 0 0 0 1 0 0 0
Эксцентриситеты:
e(0)=2
e(1)=2
e(2)=2
e(3)=2
e(4)=2
e(5)=3
e(6)=2
e(7)=3
Радиус графа = 2
Центр графа: { 0 1 2 3 4 6 }
Диаметр графа = 3
Диаметральные вершины: { 5 7 }
```

Рис. 8: Вывод матрицы смежности и метрик

На рис. 9 представлен вывод сохранённый весовой матрицы.

Ваш выбор: 5
Сохранённая весовая матрица:

ВЕСОВАЯ МАТРИЦА

	0	1	2	3	4	5	6	7
0	0	0	0	8	0	0	5	9
1	0	0	0	7	9	0	0	0
2	0	0	0	0	9	0	9	0
3	8	7	0	0	3	10	8	0
4	0	9	9	3	0	0	8	10
5	0	0	0	10	0	0	10	0
6	5	0	9	8	8	10	0	0
7	9	0	0	0	10	0	0	0

Рис. 9: Вывод весовой матрицы

На рис. 10 показан метод Шимбелла.

Ваш выбор: 6
Введите максимальное число рёбер в пути (0 - только диагональ, -1 для отмены): 4
1 - минимальные пути, 2 - максимальные, 3 - оба (или -1 для отмены): 3

Минимальные пути:

20	21	24	19	17	24	22	26
21	20	24	18	16	23	21	25
24	24	24	18	23	29	23	25
19	18	18	12	22	24	17	19
17	16	23	22	12	19	17	25
24	23	29	24	19	26	24	31
22	21	23	17	17	24	20	24
26	25	25	19	25	31	24	26

Максимальные пути:

38	35	37	36	39	38	37	34
35	38	38	37	35	37	37	39
37	38	38	37	37	39	37	39
36	37	37	40	38	38	40	38
39	35	37	38	40	38	38	36
38	37	39	38	38	40	38	38
37	37	37	40	38	38	40	38
34	39	39	38	36	38	38	40

Рис. 10: Метод Шимбелла

На рис. 11 представлен вывод количества маршрутов между заданными вершинами.

```
Ваш выбор: 7
Стартовая вершина (или -1 для отмены): 2
Конечная вершина (или -1 для отмены): 7
Количество маршрутов: 3
```

Рис. 11: Поиск количества маршрутов

На рис. 12 приведён поиск точек сочленения в неориентированном графе.

```
Ваш выбор: 8
Текущий неориентированный граф (матрица смежности):
0 1 1 1 1 1 0 1
1 0 1 1 1 1 0 1
1 1 0 1 1 0 0 1
1 1 1 0 1 1 0 1
1 1 1 1 0 1 0 1
1 1 0 1 1 0 0 1
0 0 0 0 0 0 0 1
1 1 1 1 1 1 1 0
Точки сочленения: 7
```

Рис. 12: Поиск точек сочленения

На рис. 13 показан алгоритм Дейкстры.

```
Ваш выбор: 9
Стартовая вершина (или -1 для отмены): 0
Конечная вершина (или -1 для отмены): 5
Вектор расстояний (Дейкстра):
dist[0] = 0
dist[1] = 13
dist[2] = 6
dist[3] = 9
dist[4] = 7
dist[5] = 14
dist[6] = INF
dist[7] = 6
Кратчайший путь 0 -> 5: 0 -> 2 -> 5
Длина пути: 14
Количество итераций: 120
```

Рис. 13: Вывод результатов алгоритма Дейкстры

На рис. 14 представлена генерация матриц пропускных способностей и стоимостей.

```
Ваш выбор: 10
Матрицы пропускных способностей и стоимостей сгенерированы по
распределению Чампернауна (с разными параметрами).
```

Рис. 14: Генерация матриц пропускных способностей и стоимостей

На рис. 15 показан вывод матриц пропускных способностей и стоимостей.

Ваш выбор: 11

МАТРИЦА ПРОПУСКНЫХ СПОСОБНОСТЕЙ								
	0	1	2	3	4	5	6	7
0	0	0	0	0	2	2	0	0
1	0	0	0	0	0	0	3	0
2	0	0	0	0	0	0	0	0
3	0	0	0	0	4	0	0	2
4	0	0	0	0	0	0	0	0
5	0	3	0	0	0	0	5	0
6	0	0	0	0	0	0	0	0
7	1	5	2	0	5	3	0	0

МАТРИЦА СТОИМОСТЕЙ								
	0	1	2	3	4	5	6	7
0	0	0	0	0	4	4	0	0
1	0	0	0	0	0	0	7	0
2	0	0	0	0	0	0	0	0
3	0	0	0	0	4	0	0	5
4	0	0	0	0	0	0	0	0
5	0	6	0	0	0	0	6	0
6	0	0	0	0	0	0	0	0
7	6	6	5	0	5	4	0	0

Рис. 15: Вывод матриц пропускных способностей и стоимостей

На рис. 16 представлен поиск максимального потока.

Ваш выбор: 12
 Источник (s) (или -1 для отмены): 3
 Сток (t) (или -1 для отмены): 6
 Максимальный поток из 3 в 6 = 2

Рис. 16: Поиск максимального потока

На рис. 17 показан поиск потока минимальной стоимости.

```
Ваш выбор: 13
Источник (s) (или -1 для отмены): 3
Сток (t) (или -1 для отмены): 6
Максимальный поток из 3 в 6 = 2
Требуемый объём потока для минимальной стоимости: 1 (  $\lceil 2/3 * \max \rceil$  )
Достигнутый поток: 1
Минимальная стоимость этого потока: 15

Найдено маршрутов: 1
Суммарный поток по маршрутам: 1
Суммарная стоимость по маршрутам: 15

Использованные маршруты:
3 -> 7 -> 5 -> 6 : 1 * 15 = 15
```

Рис. 17: Поиск потока минимальной стоимости

На рис. 18 показан поиск количества остовных деревьев по теореме Кирхгофа для неориентированного графа.

```
Ваш выбор: 14
Число остовных деревьев (по теореме Кирхгофа): 144
```

Рис. 18: Теорема Кирхгофа

На рис. 19 представлено построение минимального остовного дерева по алгоритму Борувки.


```

Ваш выбор: 15
MST (алгоритм Борувки) построено. Суммарный вес = 48
Матрица смежности MST:
0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 1
0 1 0 0 0 1 0 0
1 0 1 1 1 1 0 0
Весовая матрица MST:
0 0 0 0 0 0 0 9
0 0 0 0 0 0 9 0
0 0 0 0 0 0 0 6
0 0 0 0 0 0 0 3
0 0 0 0 0 0 0 7
0 0 0 0 0 0 6 8
0 9 0 0 0 6 0 0
9 0 6 3 7 8 0 0

```

Рис. 19: Построение минимального остовного дерева

На рис. 20 показано кодирование и декодирование минимального остовного дерева кодом Прюфера.

Ваш выбор: 16
Кодирование MST в код Прюфера (с сохранением весов)...
Код Прюфера (вершины): 7 6 7 7 7 5
Соответствующие веса удаляемых рёбер: 9 9 6 3 7 6 8
Последнее ребро: 5 - 7 (вес 8)

Декодирование кода Прюфера обратно в дерево...

Восстановленное дерево (матрица смежности):

0	0	0	0	0	0	0	1
0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	1
0	0	0	0	0	0	1	1
0	1	0	0	0	1	0	0
1	0	1	1	1	1	0	0

Восстановленная весовая матрица:

0	0	0	0	0	0	0	9
0	0	0	0	0	0	9	0
0	0	0	0	0	0	0	6
0	0	0	0	0	0	0	3
0	0	0	0	0	0	0	7
0	0	0	0	0	0	6	8
0	9	0	0	0	6	0	0
9	0	6	3	7	8	0	0

Рис. 20: Код Прюфера

На рис. 21 представлен алгоритм минимальной раскраски графа.

```

Ваш выбор: 17
Выберите граф для раскраски: 1 - текущий граф, 2 - последний MST
(если есть) (или -1 для отмены): 1
Раскраска выполнена. Количество одноцветных классов: 3
Вершина 0 : цвет 3
Вершина 1 : цвет 3
Вершина 2 : цвет 2
Вершина 3 : цвет 3
Вершина 4 : цвет 2
Вершина 5 : цвет 2
Вершина 6 : цвет 1
Вершина 7 : цвет 1

```

Рис. 21: Минимальная раскраска графа

На рис. 22 показана проверка графа на эйлеровость и его дальнейшая модификация.

```

Ваш выбор: 18
Граф не эйлеров. Модификация...
Вершины с нечётной степенью (отсортированы по степени): 2 0 1 4
Добавлено ребро 2 - 0
Добавлено ребро 1 - 4
Модифицированный граф:
0 0 1 0 1 1 0 1
0 0 0 0 1 1 1 1
1 0 0 0 0 0 0 1
0 0 0 0 1 0 0 1
1 1 0 1 0 0 0 1
1 1 0 0 0 0 1 1
0 1 0 0 0 1 0 0
1 1 1 1 1 1 0 0
Эйлеров цикл в модифицированном графе: 0 -> 2 -> 7 -> 0 -> 4 -> 1
-> 5 -> 6 -> 1 -> 7 -> 3 -> 4 -> 7 -> 5 -> 0

```

Рис. 22: Эйлеровы графы

На рис. 23 представлено нахождение фундаментальной системы разрезов и их симметрическая разность.

```

Ваш выбор: 19
Фундаментальная система разрезов (всего 7 разрезов):
Разрез 1: { (0,4), (0,5), (0,7) }
Разрез 2: { (1,5), (1,6), (1,7) }
Разрез 3: { (2,7) }
Разрез 4: { (3,4), (3,7) }
Разрез 5: { (0,4), (3,4), (4,7) }
Разрез 6: { (1,5), (1,7), (5,6) }
Разрез 7: { (0,5), (1,7), (5,7) }

Введите номера разрезов (через Enter) для получения их симметрической разности.
Для завершения ввода введите -1.
Номер разреза (или -1 для завершения): 1
Номер разреза (или -1 для завершения): 4
Номер разреза (или -1 для завершения): 6
Номер разреза (или -1 для завершения): -1
Отмена операции.
Результирующий разрез: { (0,4), (0,5), (0,7), (1,5), (1,7), (3,4), (3,7), (5,6) }

```

Рис. 23: Фундаментальная система разрезов

Заключение

В ходе выполнения лабораторных работ была реализована программа, которая позволяет строить связный ациклический ориентированный граф в соответствии с распределением Чампернауна, а также выполнять над графом различные алгоритмы: поиск метрик, метод Шимбелла, подсчёт маршрутов, поиск точек сочленения, алгоритм Дейкстры, вычисление максимального потока и потока минимальной стоимости, подсчёт числа остовных деревьев по теореме Кирхгофа, построение минимального остова алгоритмом Борушки, кодирование и декодирование дерева кодом Прюфера с сохранением весов, эвристическую раскраску графа, проверку на эйлеровость и построение эйлерова цикла, а также построение фундаментальной системы разрезов и операции над разрезами. Реализована защита от некорректного пользовательского ввода.

В ходе выполнения лабораторной работы №1 реализовано формирование случайного связного ациклического ориентированного графа с задаваемым пользователем количеством вершин; степени вершин генерируются в соответствии с распределением Чампернауна. Вычислены эксцентриситеты вершин, центр графа и диаметральные вершины. Реализован метод Шимбелла для нахождения минимальных и максимальных путей с ограничением по числу рёбер. Добавлена возможность подсчёта количества маршрутов между двумя заданными вершинами.

В лабораторной работе №2 для сгенерированных графов реализован поиск точек сочленения в неориентированном графе с помощью обхода в глубину. Реализован классический алгоритм Дейкстры для нахождения кратчайшего пути между выбранными вершинами (с проверкой на отрицательные веса).

В лабораторной работе №3 на основе ориентированного графа сгенерированы матрицы пропускных способностей и стоимостей. Найден максимальный поток алгоритмом Эдмондса–Карпа. Вычислен поток минимальной стоимости заданной величины с использованием алгоритма Дейкстры на приведённых стоимостях.

В лабораторной работе №4 для неориентированного графа вычислено число остовных деревьев с помощью матричной теоремы Кирхгофа. Построен минимальный остов алгоритмом Борушки, выполнено его кодирование в код Прюфера с сохранением весов рёбер и последующее декодирование. Реализован улучшенный алгоритм для нахождения минимальной раскраски графа.

В лабораторной работе №5 проверено, является ли граф эйлеровым; при необходимости выполнена модификация графа для достижения чётности степеней всех вершин, после чего построен эйлеров цикл. На основе минимального остовного дерева построена фундаментальная система разрезов, реализована операция симметрической разности для получения произвольных разрезов по выбору пользователя.

Достоинства

- реализация в объектно-ориентированном стиле: удобное разделение функционала (каждое задание в отдельном модуле);
- раскраска графа реализована при помощи улучшенного алгоритма последовательного раскрашивания (жадный алгоритм с эвристикой: порядок вершин отсортирован по убыванию их степени);
- логирование изменений при модификации графа в эйлеров;

Недостатки

- отсутствие возможности задать параметры распределения Чампернауна пользователем;
- В программе существуют похожие по структуре функции, например, генерация различных матриц, которые можно оптимизировать и объединить.
- отсутствие возможности выбора ориентированности или её отсутствия при генерации графа;

Масштабирование программы

- добавление новых методов и алгоритмов для работы с графом, например, алгоритм Дейкстры для отрицательных весов;
- добавление других распределений для генерации;
- добавление возможности задания параметров распределения пользователем;

Список литературы

Список литературы

- [1] Секция «Телематика». — Текст : электронный // tema.srbstu.ru : [сайт]. — URL: <https://tema.srbstu.ru/tgraph/> (дата обращения: 17.05.2025).
- [2] Новиков Ф.А. ДИСКРЕТНАЯ МАТЕМАТИКА ДЛЯ ПРОГРАММИСТОВ / Ф.А. Новиков.— 3-е издание.— Питер : Питер Пресс, 2009.— 384 с.
- [3] Вадзинский Р.Н. Справочник по вероятностным распределениям.- Санкт-Петербург: Наука, 2001.- 295 с.